CS336 作业 1（basics）：构建 Transformer LM

Version 26.0.3

CS336 Staff

Spring 2026

1 作业概览

在本次作业中，你将从零开始构建训练一个标准 Transformer 语言模型（LM）所需的全部组件，并训练若干模型。

你将实现

1. 字节对编码（BPE）分词器（第 2 节）
2. Transformer 语言模型（LM）（第 3 节）
3. 交叉熵损失函数和 AdamW 优化器（第 4 节）
4. 训练循环，并支持序列化与加载模型状态和优化器状态（第 5 节）

你将运行

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器对数据集进行编码，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer LM。
4. 使用训练好的 Transformer LM 进行样本生成，并评估困惑度。
5. 在 OpenWebText 上训练模型，并将你达到的困惑度提交到 leaderboard。

你可以使用什么

我们期望你从零开始构建每个组件。特别地,除以下内容外,你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如 `Module`、`ModuleList`、`Sequential` 等）。[1]
- `torch.optim.Optimizer` 基类

你可以使用任何其他 PyTorch 定义。如果你想使用某个函数或类，但不确定是否允许使用，请随时在 Slack 上提问。如果存疑，请思考使用它是否会破坏本作业“from-scratch”的精神。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分。这会使你更难深入参与并从课程材料中学习。

允许使用 AI 工具回答高层概念问题，或提供低层编程文档，例如函数签名和库 API。然而，不允许使用 AI 工具来实现任何作业的任何部分。这包括编码 agent（例如 Cursor Agents、Codex、Claude Code）和 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI agent 时，请确保它使用所提供的 `AGENTS.md` 文件。使用聊天机器人时，也应包含 prompt。

[1] 完整列表见 https://pytorch.org/docs/stable/nn.html#containers。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot），不过非 AI 自动补全（例如自动补全函数名）完全可以使用。以往学生强调，关闭 AI 自动补全使他们更容易深入参与课程材料。

完整的 AI Policy 见此文档。

代码看起来是什么样的

作业代码以及本说明文档可在 GitHub 获取：

github.com/stanford-cs336/assignment1-basics

请 `git clone` 该仓库。如果有任何更新，我们会通知你，以便你可以 `git pull` 获取最新版本。

1. `cs336_basics/*`：这里是你编写代码的地方。注意此目录中没有代码——你可以完全从零开始自由实现！
2. `adapters.py`：这里有一组你的代码必须具备的功能。对于每个功能（例如 scaled dot product attention），请填写其实现（例如 `run_scaled_dot_product_attention`），方式是简单调用你自己的代码。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；它只是 glue code。
3. `test_*.py`：这里包含你必须通过的全部测试（例如 `test_scaled_dot_product_attention`），这些测试会调用 `adapters.py` 中定义的 hooks。不要编辑测试文件。

如何提交

为了提交，请运行 `make_submission.sh` 来构造提交 zip 文件。如果你有大型数据文件或不想包含在提交 zip 中的检查点，请确保将任何额外文件加入脚本中的排除列表。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请排版你的回答。
- `code.zip`：包含你编写的所有代码。

要提交到 leaderboard，请向以下仓库提交 PR：

github.com/stanford-cs336/assignment1-basics-leaderboard

详细提交说明见 leaderboard 仓库中的 `README.md`。

从哪里获取数据集

本作业将使用两个预处理数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。两个数据集都是单个大型纯文本文件。

如果你正在跟班完成作业，可以在 compute guide 中找到下载数据集的说明。如果你是在家自学，可以使用 `README.md` 中的命令下载这些文件。

Low-Resource Tip: Init

在课程的作业讲义中，我们会给出建议，说明如何在较少 GPU 资源或没有 GPU 资源的情况下完成部分作业。例如，我们有时会建议缩小数据集或模型规模，或解释如何在 Mac 集成 GPU 或 CPU 上运行训练代码。你会在蓝色框中看到这些“低资源提示”（就像这个框）。即使你是已注册的 Stanford 学生并可访问课程机器，这些提示也可能帮助你更快迭代并节省时间，因此建议阅读！

Low-Resource Tip: Assignment 1 on Apple Silicon or CPU

使用 staff solution code，我们可以在配备 36 GB RAM 的 Apple M4 Max 芯片上训练一个 LM，使其生成相当流畅的文本：在 Metal GPU（MPS）上不到 5 分钟，使用 CPU 约 30 分钟。如果这些词对你来说没有太多意义，不必担心！只需知道，如果你拥有一台较新的笔记本电脑，并且你的实现正确且高效，你将能够训练一个小型 LM，使其以尚可的流畅度生成简单儿童故事。在作业后面，我们会解释如果你使用 CPU 或 MPS 需要做哪些修改。

2 字节对编码（BPE）分词器

在作业的第一部分，我们将训练并实现一个字节级字节对编码（BPE）分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体而言，我们会将任意（Unicode）字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。稍后，我们会使用该分词器将文本（字符串）编码为 tokens（整数序列），用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它将字符映射到整数码点。截至 Unicode 17.0（2025 年 9 月发布），该标准定义了横跨 172 种文字系统的 159,801 个字符。例如，字符“s”的码点为 115（通常记作 U+0073，其中 U+ 是惯例前缀，0073 是 115 的十六进制表示），字符“牛”的码点为 29275。在 Python 中，可以使用 `ord()` 函数将单个 Unicode 字符转换为其整数表示。`chr()` 函数则将整数 Unicode 码点转换为包含相应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

Problem (unicode1): Understanding Unicode (1 point)

(a) `chr(0)` 返回哪个 Unicode 字符？

交付内容：一句回答。

(b) 这个字符的字符串表示（`__repr__()`）与其打印表示有何不同？

交付内容：一句回答。

(c) 当这个字符出现在文本中时会发生什么？在你的 Python 解释器中尝试以下内容并看看它是否符合你的预期，可能会有所帮助：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付内容：一句回答。

2.2 Unicode 编码

虽然 Unicode 标准定义了从字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不实际，因为词表会大到难以承受（约 150K 项）且非常稀疏（许多字符非常罕见）。相反，我们将使用 Unicode 编码，它会把一个 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网的主导编码（超过 98% 的网页使用）。

要将 Unicode 字符串编码为 UTF-8，可以使用 Python 中的 `encode()` 函数。要访问 Python `bytes` 对象底层的字节值，可以对其进行迭代（例如调用 `list()`）。最后，可以使用 `decode()` 函数将 UTF-8 字节串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

通过将 Unicode 码点转换为字节序列（例如通过 UTF-8 编码），我们本质上是把一串码点（21 位整数，其中有 159,801 个有效值）转换成一串字节值（范围为 0 到 255 的整数）。长度为 256 的字节词表更容易处理。使用字节级分词时，我们无需担心词表外 token，因为我们知道任何输入文本都可以表示为 0 到 255 的整数序列。

Problem (unicode2): Unicode Encodings (3 points)

(a) 相比 UTF-16 或 UTF-32，有哪些理由更倾向于在 UTF-8 编码字节上训练分词器？比较这些编码在不同输入字符串上的输出可能会有所帮助。

交付内容： 一到两句回答。

(b) 考虑下面这个（错误的）函数，它意图将 UTF-8 字节串解码为 Unicode 字符串。为什么这个函数是错误的？请给出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

交付内容： 一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串示例，并用一句话解释为什么该函数是错误的。

(c) 给出一个不能解码为任何 Unicode 字符的双字节序列。

交付内容： 一个示例，并用一句话解释。

2.3 子词分词

字节级分词可以缓解词级分词器面临的词表外问题，但把文本分成字节会导致输入序列极长。这会减慢模型训练，因为一个包含 10 个单词的句子在词级语言模型中可能只有 10 个 token，但在字符级模型中可能有 50 个或更多 token（取决于单词长度）。处理这些更长的序列会使模型每一步需要更多计算。此外，在字节序列上做语言建模也很困难，因为更长的输入序列会在数据中形成长期依赖。

子词分词位于词级分词器和字节级分词器之间。注意，字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器用更大的词表换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 经常出现在原始训练文本中，那么将其作为词表中的一个条目就能把这个 3-token 序列压缩成单个 token。

我们如何选择要添加到词表中的这些子词单元？R. Sennrich et al. [3] 提出使用 byte-pair encoding（BPE；P. Gage [5]），这是一种压缩算法，会迭代地将最频繁的字节对替换（“merge”）为一个新的、未使用的索引。注意，该算法向词表添加子词 token 的目标是最大化输入序列的压缩；如果某个单词在输入文本中出现次数足够多，它就会被表示为单个子词单元。

通过 BPE 构造词表的子词分词器通常称为 BPE 分词器。在本作业中，我们将实现字节级 BPE 分词器，其中词表项是字节或合并后的字节序列，这在处理词表外问题和保持可管理的输入序列长度方面兼具两者优势。构建 BPE 分词器词表的过程称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器训练过程包括三个主要步骤。

词表初始化

分词器词表是从字节串 token 到整数 ID 的一对一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是所有字节的集合。由于共有 256 个可能的字节值，因此初始词表大小为 256。

预分词

拥有词表后，原则上你可以统计文本中字节相邻出现的频率，并从最频繁的字节对开始合并。然而，这在计算上非常昂贵，因为每次合并都需要完整遍历语料。此外，直接跨整个语料合并字节可能会产生仅在标点上不同的 token（例如 `dog!` 与 `dog.`）。这些 token 会得到完全不同的 token ID，尽管它们很可能具有很高的语义相似性（因为它们只在标点上不同）。

为避免这种情况，我们会对语料进行预分词。你可以把它看作对语料的一种粗粒度分词，它帮助我们统计字符对出现的频率。例如，单词 `'text'` 可能是一个出现 10 次的 pre-token。在这种情况下，当我们统计字符‘t’和‘e’相邻出现的次数时，会看到单词 `'text'` 中‘t’和‘e’相邻，于是可以将它们的计数增加 10，而无需逐字符扫描整个语料。由于我们训练的是字节级 BPE 模型，每个 pre-token 都表示为 UTF-8 字节序列。

R. Sennrich et al. [3] 的原始 BPE 实现通过简单地按空白切分来预分词（即 `s.split(" ")`）。这种方法仍可见于基于 SentencePiece 的分词器（例如 Llama 1 和 2 的分词器）。

大多数现代分词器使用基于正则表达式的预分词器，这一实践来自 GPT-2；A. Radford et al. [6]。我们将使用原始正则表达式的一个稍微更漂亮的形式，取自 github.com/openai/tiktoken/pull/234/files：

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

交互式地用这个预分词器切分一些文本，可能有助于你更好地理解它的行为：

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

不过，在你的代码中，应使用 `re.finditer`，以避免在构造从 pre-token 到其计数的映射时存储所有预分词后的单词。

计算 BPE merges

现在我们已经把输入文本转换成了 pre-token，并将每个 pre-token 表示为 UTF-8 字节序列，就可以计算 BPE merges（即训练 BPE 分词器）。从高层看，BPE 算法会迭代地统计每个字节对，并找出频率最高的字节对（“A”，“B”）。该最频繁字节对（“A”，“B”）的每次出现都会被合并，即替换成一个新的 token “AB”。这个新的合并 token 会被添加到词表中；因此，BPE 训练后的最终词表大小等于初始词表大小（在我们的情形中是 256）加上训练期间执行的 BPE merge 操作数量。为提高 BPE 训练效率，我们不考虑跨越 pre-token 边界的字节对。[2] 计算 merges 时，如果字节对频率出现并列，请确定性地选择字典序更大的 pair。例如，如果 pairs (“A”，“B”)、(“A”，“C”)、(“B”，“ZZ”) 和 (“BA”，“A”) 都有最高频率，我们会合并 (“BA”，“A”):

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token

某些字符串（例如 `<|endoftext|>`）常用于编码元数据（例如文档之间的边界）。编码文本时，通常希望将某些字符串视为“特殊 token”，它们永远不应被拆成多个 token（即总是作为单个 token 保留）。例如，序列结束字符串 `<|endoftext|>` 应始终作为单个 token（即单个整数 ID）保留，以便我们知道何时停止语言模型的生成。这些特殊 token 必须添加到词表中，因此它们有对应的固定 token ID。

R. Sennrich et al. [3] 的 Algorithm 1 包含一个低效的 BPE 分词器训练实现（本质上遵循我们上面概述的步骤）。作为第一个练习，实现并测试这个函数可能有助于检查你的理解。

示例 (bpe_example)：BPE 训练示例

下面是 R. Sennrich et al. [3] 中的一个风格化示例。考虑由以下文本组成的语料：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

并且词表有一个特殊 token `<|endoftext|>`。

词表

我们用特殊 token `<|endoftext|>` 和 256 个字节值初始化词表。

预分词

为简单起见并聚焦于 merge 过程，本例假设预分词只是按空白切分。当我们预分词并计数后，会得到频率表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

[2] 注意，R. Sennrich et al. [3] 的原始 BPE 形式规定要包含一个 end-of-word token。训练字节级 BPE 模型时，我们不会添加 end-of-word-token，因为所有字节（包括空白和标点）都已经包含在模型词表中。由于我们显式表示空格和标点，学习到的 BPE merges 会自然反映这些单词边界。

将其表示为 dict[tuple[bytes, ...], int] 会很方便，例如 (l,o,w): 5, ...。注意，即使是单个字节，在 Python 中也是一个 bytes 对象。Python 中没有用于表示单个字节的 byte 类型，就像 Python 中没有表示单个字符的 char 类型一样。

合并

我们首先查看每个连续的字节对，并对它们出现所在单词的频率求和：lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6。pairs ('e', 's') 和 ('s', 't') 并列，因此取字典序更大的 pair，即 ('s', 't')。然后我们会合并 pre-token，得到 (l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6。

第二轮中，我们看到 (e, st) 是最常见的 pair（计数为 9），于是合并为 (l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6。继续进行下去，最终得到的 merge 序列为 ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']。

如果只取 6 次 merge，则有 ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']，词表元素为 [<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]。

使用这个词表和 merge 集合，单词 newest 会被分词为 [ne, west]。

2.5 BPE 分词器训练实验

现在让我们在 TinyStories 数据集上训练一个字节级 BPE 分词器。查找/下载数据集的说明见第 1 节。开始之前，我们建议你先查看 TinyStories 数据集，了解数据中包含什么。

并行化预分词

你会发现一个主要瓶颈是预分词步骤。你可以使用内置库 multiprocessing 对代码进行并行化，从而加速预分词。具体来说，在并行预分词实现中，我们建议你对语料分块，同时确保分块边界位于一个特殊 token 的开头。你可以逐字使用以下链接中的 starter code 来获取分块边界，然后利用这些边界将工作分配给各个进程：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块始终是有效的，因为我们永远不希望跨文档边界进行合并。就本作业而言，你总是可以用这种方式切分。不必担心收到一个不包含 <|endoftext|> 的超大语料这一边界情况。

预分词前移除特殊 token

在使用正则表达式模式运行预分词（使用 re.finditer）之前，应从语料（或并行实现中的 chunk）中剥离所有特殊 token。确保你按特殊 token 进行切分，使其分隔的文本之间不会发生合并。例如，如果你有一个语料（或 chunk）形如 [Doc 1]<|endoftext|>[Doc 2]，就应按特殊 token <|endoftext|> 切分，并分别对 [Doc 1] 和 [Doc 2] 做预分词，从而不会跨文档边界进行合并。换句话说，特殊 token 在训练期间定义硬分段边界，但它们本身不应参与 merge 计数。可以使用 re.split，将 "|".join(special_tokens) 作为分隔符来完成这一点（需要谨慎使用 re.escape，因为特殊 token 中可能出现 |）。测试 test_train_bpe_special_tokens 会检查这一点。

优化 merge 步骤

上面风格化示例中的 BPE 训练朴素实现很慢，因为每次 merge 都会遍历所有字节对来识别最频繁的 pair。然而，每次 merge 后唯一会变化的 pair 计数，是那些与被合并 pair 重叠的 pair。因此，可以通过索引所有 pair 的计数并增量更新这些计数来提升 BPE 训练速度，而不是显式遍历每个字节对来计数 pair 频率。这个缓存过程可以带来显著加速，但我们也指出，BPE 训练中的 merge 部分在

Python 中无法并行化。

Low-Resource Tip: Profiling

你应该使用 `cProfile` 或 `py-spy` 等 profiling 工具来识别实现中的瓶颈，并集中优化这些瓶颈。

Low-Resource Tip: ”Downscaling”

不要一开始就把分词器训练放在完整 TinyStories 数据集上，我们建议你先在数据的一个小子集——“debug dataset”——上训练。例如，你可以改为在 TinyStories 验证集上训练分词器，该集合有 22K 个文档，而不是 2.12M 个。只要可能就进行 downscaling 是一种加快开发速度的通用策略：例如使用更小的数据集、更小的模型规模等。选择 debug 数据集大小或超参数配置需要仔细考虑：你希望 debug 集足够大，以便具有与完整配置相同的瓶颈（这样你的优化可以泛化），但又不能太大而导致运行时间过长。

Problem (train_bpe): BPE Tokenizer Training (15 points)

交付内容：编写一个函数，给定输入文本文件路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应至少处理以下输入参数：
输入

`input_path: str`
含有 BPE 分词器训练数据的文本文件路径。

`vocab_size: int`
正整数，定义最终词表的最大大小（包括初始字节词表、merge 产生的词表项以及任何特殊 token）。

`special_tokens: list[str]`
要添加到词表中的字符串列表。训练期间，将它们视为硬边界，阻止跨其跨度进行 merge，但在计算 merge 统计时不包含它们。

你的 BPE 训练函数应返回得到的词表和 merges：
输出

`vocab: dict[int, bytes]`
分词器词表，是从 `int`（词表中的 token ID）到 `bytes`（token bytes）的映射。

`merges: list[tuple[bytes, bytes]]`
训练产生的 BPE merges 列表。列表中的每一项都是 bytes 元组（`<token1>`, `<token2>`），表示 `<token1>` 与 `<token2>` 被合并。merges 应按创建顺序排列。

为了用我们提供的测试来测试你的 BPE 训练函数，你首先需要实现 `[adapters.run_train_bpe]` 中的 test adapter。然后运行 `uv run pytest tests/test_train_bpe.py`。你的实现应能通过所有测试。可选地（这可能会投入大量时间），你可以用某种系统语言实现训练方法的关键部分，例如 C++（考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`）。如果这样做，请注意哪些操作需要复制、哪些可以直接从 Python 内存读取，并确保留下构建说明，或确保仅用 `pyproject.toml` 即可构建。还需注意，GPT-2 正则表达式在大多数 regex engine 中支持不好，并且在多数支持它的引擎中也会过慢。我们验证过 Oniguruma 相当快且支持 negative lookahead，但 Python 中的 `regex` 包甚至可能更快。

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

(a) 在 TinyStories 数据集上训练一个字节级 BPE 分词器，最大词表大小为 10,000。确保将 TinyStories 的 `<|endoftext|>` 特殊 token 添加到词表中。将得到的词表和 merges 序列化到磁盘以供进一步检查。训练花费了多少时间和内存？词表中最长的 token 是什么？它是否合理?

资源要求： $\leq$ 30 分钟（无 GPU），$\leq$ 30 GB RAM

提示 使用预分词期间的 multiprocessing 以及以下两个事实，你应该能够在 2 分钟内完成 BPE 训练：

(a) 数据文件中的 `<|endoftext|>` token 分隔文档。

(b) 在应用 BPE merges 之前，`<|endoftext|>` token 会作为特殊情况处理。

交付内容： 一到两句回答。

(b) Profile 你的代码。分词器训练过程中哪个部分耗时最多?

交付内容： 一到两句回答。

接下来，我们将尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。和前面一样，我们建议先查看数据集，以便更好地理解其内容。

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

(a) 在 OpenWebText 数据集上训练一个字节级 BPE 分词器，最大词表大小为 32,000。将得到的词表和 merges 序列化到磁盘以供进一步检查。词表中最长的 token 是什么？它是否合理?

资源要求： $\leq$ 12 小时（无 GPU），$\leq$ 100 GB RAM

交付内容： 一到两句回答。

(b) 比较并对照在 TinyStories 与 OpenWebText 上训练得到的分词器。

交付内容： 一到两句回答。

2.6 BPE 分词器：编码与解码

在作业上一部分中，我们实现了一个函数，在输入文本上训练 BPE 分词器，以得到分词器词表和 BPE merges 列表。现在，我们将实现一个 BPE 分词器，它加载给定词表和 merges 列表，并使用它们将文本编码为 token ID 或从 token ID 解码为文本。

2.6.1 编码文本

BPE 编码文本的过程与训练 BPE 词表的方式相呼应。主要有几个步骤。

步骤 1：预分词。我们首先对序列进行预分词，并将每个 pre-token 表示为 UTF-8 字节序列，就像 BPE 训练中所做的一样。我们将在每个 pre-token 内把这些字节 merge 成词表元素，独立处理每个 pre-token（不跨 pre-token 边界进行 merge）。

步骤 2：应用 merges。然后，我们取 BPE 训练期间创建的词表元素 merge 序列，并按创建顺序将其应用到我们的 pre-token 上。

示例 (bpe_encoding)：BPE 编码示例

例如，假设输入字符串为 `'the cat ate'`，词表为 `0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'`，学习到的 merges 为 `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]`。首先，预分词器会将该字符串切分为 `['the', ' cat', ' ate']`。然后，我们会查看每个 pre-token 并应用 BPE merges。

第一个 pre-token `'the'` 初始表示为 `[b't', b'h', b'e']`。查看 merges 列表，我们识别出第一个可应用的 merge 是 `(b't', b'h')`，并用它将 pre-token 转换为 `[b'th', b'e']`。随后，我们回到 merges 列表，识别出下一个可应用 merge 为 `(b'th', b'e')`，它将 pre-token 转换为 `[b'the']`。最后，再次查看 merges 列表，我们发现没有更多适用于该字符串的 merge（因为整个 pre-token 已合并成单个 token），因此完成 BPE merges 的应用。对应的整数序列为 `[9]`。

对其余 pre-token 重复此过程，我们会看到 pre-token `' cat'` 在应用 BPE merges 后表示为 `[b' c', b'a', b't']`，变成整数序列 `[7, 1, 5]`。最后一个 pre-token `' ate'` 在应用 BPE merges 后为 `[b' at', b'e']`，变成整数序列 `[10, 3]`。因此，对输入字符串进行编码的最终结果为 `[9, 7, 1, 5, 10, 3]`。

特殊 token

你的分词器在编码文本时应能正确处理用户定义的特殊 token（在构造分词器时提供）。

内存考虑

假设我们想分词一个无法装入内存的大型文本文件。为了高效地分词这个大文件（或任何其他数据流），我们需要将其切成可管理的 chunk，并依次处理每个 chunk，使内存复杂度保持常数，而不是随文本大小线性增长。这样做时，需要确保 token 不会跨越 chunk 边界，否则得到的分词结果将不同于把整个序列一次性加载到内存中进行分词的朴素方法。

2.6.2 解码文本

要将整数 token ID 序列解码回原始文本，可以简单地查找每个 ID 在词表中对应的条目（一个字节序列），将它们拼接在一起，然后把 bytes 解码为 Unicode 字符串。注意，输入 ID 并不保证映射到有效的 Unicode 字符串（因为用户可以输入任意整数 ID 序列）。如果输入 token ID 不产生有效 Unicode 字符串，应将 malformed bytes 替换为官方 Unicode replacement character U+FFFD。[3] `bytes.decode` 的 `errors` 参数控制如何处理 Unicode 解码错误，使用 `errors='replace'` 会自动将 malformed data 替换为 replacement marker。

Problem (tokenizer): Implementing the tokenizer (15 points)

交付内容：实现一个 `Tokenizer` 类，给定词表和 merges 列表，将文本编码为整数 ID，并将整数 ID 解码为文本。你的分词器还应支持用户提供的特殊 token（如果它们尚未在词表中，则追加到词表）。我们推荐以下接口：

`def __init__(self, vocab, merges, special_tokens=None)` 从给定词表、merges 列表以及（可选的）特殊 token 列表构造分词器。此函数应接受以下参数：

- `vocab: dict[int, bytes]`
- `merges: list[tuple[bytes, bytes]]`

[3] 更多关于 Unicode replacement character 的信息，见 https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character。

- `special_tokens: list[str] | None = None`

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)` 类方法，从序列化的词表和 merges 列表（格式与你的 BPE 训练代码输出相同）以及（可选的）特殊 token 列表构造并返回一个 `Tokenizer`。此方法应接受以下额外参数：

- `vocab_filepath: str`
- `merges_filepath: str`
- `special_tokens: list[str] | None = None`

`def encode(self, text: str) -> list[int]` 将输入文本编码为 token ID 序列。

`def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]` 给定字符串 iterable（例如 Python 文件句柄），返回一个惰性产生 token ID 的 generator。这是对无法直接加载入内存的大文件进行内存高效分词所必需的。

`def decode(self, ids: list[int]) -> str` 将 token ID 序列解码为文本。

为了用我们提供的测试来测试你的 `Tokenizer`，你首先需要实现 `[adapters.get_tokenizer]` 中的 test adapter。然后运行 `uv run pytest tests/test_tokenizer.py`。你的实现应能通过所有测试。

2.7 实验

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

(a) 从 TinyStories 和 OpenWebText 中各采样 10 个文档。使用你之前训练好的 TinyStories 和 OpenWebText 分词器（词表大小分别为 10K 和 32K），将这些采样文档编码为整数 ID。每个分词器的压缩率（bytes/token）是多少?

交付内容： 一到两句回答。

(b) 如果用 TinyStories 分词器对你的 OpenWebText 样本进行分词，会发生什么？比较压缩率和/或定性描述发生的情况。

交付内容： 一到两句回答。

(c) 估计你的分词器吞吐量（例如 bytes/second）。对 Pile 数据集（825GB 文本）进行分词需要多久?

交付内容： 一到两句回答。

(d) 使用你的 TinyStories 和 OpenWebText 分词器，将各自的训练集与开发集编码为整数 token ID 序列。稍后我们会用它来训练语言模型。我们建议将 token ID 序列序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择?

交付内容： 一到两句回答。

3 Transformer 语言模型架构

语言模型的输入是一个批处理的整数 token ID 序列（即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`），并返回词表上的（批处理）归一化概率分布（即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch Tensor），其中预测分布是针对每个输入 token 的

下一个词。在训练语言模型时，我们使用这些下一词预测来计算真实下一词与预测下一词之间的交叉熵损失。在推理期间用语言模型生成文本时，我们取最后一个时间步（即序列中的最后一项）的预测下一词分布来生成序列中的下一个 token（例如取概率最高的 token、从分布中采样等），将生成的 token 加到输入序列中，然后重复。

在本作业的这一部分，你将从零开始构建这个 Transformer 语言模型。我们会先给出模型的高层描述，然后逐步详细说明各个组件。

3.1 Transformer LM

给定 token ID 序列，Transformer 语言模型使用输入嵌入将 token ID 转换为 dense vectors，将嵌入后的 tokens 传入 `num_layers` 个 Transformer blocks，然后应用一个学习到的线性投影（“output embedding”或“LM head”）来生成预测的 next-token logits。图 1 给出了示意图。

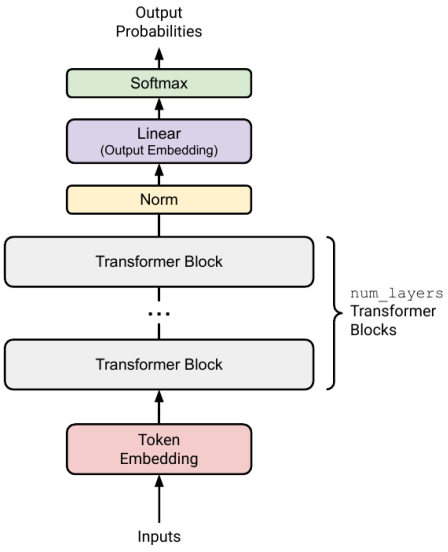


图 1: 我们的 Transformer 语言模型概览。

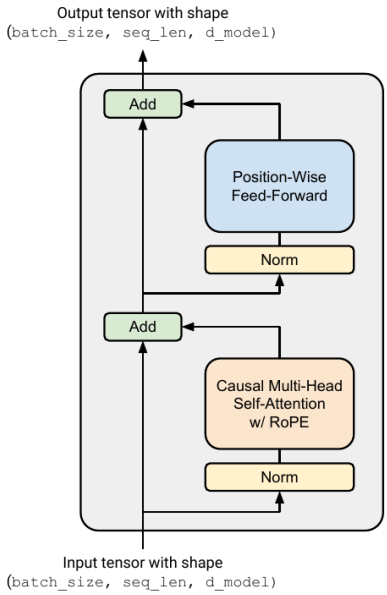


图 2: pre-norm Transformer 块。

图中文字翻译： Figure 1: Output Probabilities = 输出概率；Softmax = Softmax；Linear (Output Embedding) = 线性层（输出嵌入）；Norm = 归一化；Transformer Block = Transformer 块；num_layers Transformer Blocks = `num_layers` 个 Transformer 块；Token Embedding = Token 嵌入；Inputs = 输入。Figure 2: Output tensor with shape = 输出张量形状；Input tensor with shape = 输入张量形状；Add = 相加；Position-Wise Feed-Forward = 按位置前馈网络；Causal Multi-Head Self-Attention w/ RoPE = 带 RoPE 的因果多头自注意力；Norm = 归一化。

词元嵌入

在第一步中，Transformer 将（批处理的）token ID 序列嵌入为一个向量序列，这些向量包含 token 身份信息（图 1 中的红色块）。

更具体地说，给定 token ID 序列，Transformer 语言模型使用 token embedding 层生成向量序列。每个 embedding 层接收一个形状为 `(batch_size, sequence_length)` 的整数张量，并生成形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

Pre-norm Transformer 块

嵌入之后，activations 会由多个结构相同的神经网络层处理。一个标准 decoder-only Transformer 语言模型由 `num_layers` 个相同层组成（通常称为 Transformer “blocks”）。每个 Transformer block 接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回形状为 `(batch_size, sequence_length, d_model)` 的输出。每个 block 会在序列上聚合信息（通过 self-attention），并对其进行非线性变换（通过 feed-forward layers）。

经过 `num_layers` 个 Transformer blocks 后，我们会将最终 activations 转换为词表上的分布。

我们将实现“pre-norm”Transformer block（第 3.4 节详细说明），它还要求在最后一个 Transformer block 之后使用 layer normalization（下文详述），以确保输出具有合适的缩放。

在这个 normalization 之后，我们将使用一个标准的学习型线性变换，把 Transformer blocks 的输出转换为预测 next-token logits（参见 A. Radford et al. [7] 公式 2）。

3.2 备注：Batching、Einsum 与高效计算

在整个 Transformer 中，我们会对许多类似 batch 的输入执行相同计算。下面是几个例子：

- batch 中的元素：我们对每个 batch element 应用相同的 Transformer forward 操作。
- 序列长度：RMSNorm 和 feed-forward 等“position-wise”操作对序列中每个位置的处理方式相同。
- attention heads：在“multi-headed”attention 操作中，attention 操作会在 attention heads 上进行 batch 化。

拥有一种符合人体工学的方式来执行这类操作非常有用：它既能充分利用 GPU，又易于阅读和理解。许多 PyTorch 操作可以在张量开头接受多余的“batch-like”维度，并高效地在这些维度上重复/广播操作。

例如，假设我们正在做一个 position-wise、batched 操作。我们有一个形状为 `(batch_size, sequence_length, d_model)` 的“数据张量” D，并希望与一个形状为 `(d_model, d_model)` 的矩阵 A 做 batched vector-matrix multiply。在这种情况下，`D @ A` 会执行 batched matrix multiply；这是 PyTorch 中的高效 primitive，其中 `(batch_size, sequence_length)` 维度会被 batch 化。

因此，假设你的函数可能获得额外的 batch-like 维度，并将这些维度保留在 PyTorch shape 的开头，会很有帮助。为了将张量组织成可用这种方式 batch 化的形式，可能需要使用多步 `view`、`reshape` 和 `transpose` 调整形状。这可能会有些麻烦，并且常常很难读懂代码在做什么以及张量的形状是什么。

更符合人体工学的选择是在 `torch.einsum` 中使用 einsum notation，或者使用 `einops` 或 `einx` 这样的框架无关库。两个关键操作是 `einsum` 和 `rearrange`：`einsum` 可以在输入张量的任意维度上做 tensor contractions；`rearrange` 可以重排、连接和拆分任意维度。事实证明，机器学习中几乎所有操作都是维度整理与 tensor contraction 的某种组合，偶尔再加上（通常逐点的）非线性函数。这意味着，使用 einsum notation 可以让大量代码更可读、更灵活。

我们强烈建议在本课程中学习并使用 einsum notation。之前没有接触过 einsum notation 的学生应使用 `einops`（文档见此处），已经熟悉 `einops` 的学生应学习更通用的 `einx`（见此处）。[4] 我们提供的环境中已经安装了这两个包。

下面给出一些 einsum notation 的使用示例。这些内容是对 `einops` 文档的补充，你应该先阅读该文档。

[4] 值得注意的是，尽管 `einops` 获得了大量支持，`einx` 还没有经过同样充分的实战检验。如果发现 `einx` 的任何限制或 bug，你可以退回到使用 `einops` 加一些普通 PyTorch。

示例 (einstein_example1)：使用 einops.einsum 进行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#                          D                A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is
↪  constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (einstein_example2)：使用 einops.rearrange 进行广播操作

我们有一批图像，对于每张图像，我们希望基于某个 scaling factor 生成 10 个变暗版本:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,    "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width
↪  channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

示例 (einstein_example3)：使用 einops.rearrange 进行像素混合

假设我们有一批图像，表示为形状 `(batch, height, width, channel)` 的张量，并希望对图像所有像素执行线性变换，但这个变换应对每个 channel 独立发生。线性变换由形状为 `(height * width, height * width)` 的矩阵 B 表示。

```
channels_last = torch.randn(64, 32, 32, 3)       # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
```

```
channels_last_transformed =
↪  channels_last_flat_transformed.view(*channels_last.shape)
```

相反，使用 `einops`：

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

或者，如果你想更激进些，可以一次性用 `einx.dot`（`einops.einsum` 的 `einx` 等价物）完成：

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

这里的第一个实现可以通过在前后添加注释来指明输入和输出形状而改进，但这很笨重且容易出 bug。使用 einsum notation，文档就是实现！

Einsum notation 可以处理任意输入 batching 维度，同时还具有自文档化的关键优势。在使用 einsum notation 的代码中，输入和输出张量的相关形状是什么会更清晰。对于其他张量，你可以考虑使用 Tensor type hints，例如使用 `jaxtyping` 库（它并不专属于 JAX）。

我们将在作业 2 中更多讨论使用 einsum notation 的性能影响；现在只需知道，它们几乎总是优于替代方案！

3.2.1 数学记号与内存顺序

许多机器学习论文在记号中使用行向量，这会得到与 NumPy 和 PyTorch 默认使用的 row-major memory ordering 很匹配的表示。使用行向量时，线性变换写作

$$y = xW^\top, \tag{1}$$

其中 row-major $W \in \mathbb{R}^{d_{out} \times d_{in}}$，行向量 $x \in \mathbb{R}^{1 \times d_{in}}$。注意，这使我们可以通过增加 x 的最外层维度来 batch 化输入，也就是说，可以用矩阵输入 $X \in \mathbb{R}^{batch \times d_{in}}$ 替代向量输入 x。

在线性代数中，更常见的是使用列向量，此时线性变换写作

$$y = Wx, \tag{2}$$

其中 row-major $W \in \mathbb{R}^{d_{out} \times d_{in}}$，列向量 $x \in \mathbb{R}^{d_{in}}$。在这种设定下要 batch 化输入，batch 维度必须放在 x 的最后，因此需要把 x 替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{in} \times batch}$。

本作业的数学记号大多使用列向量，因为数学通常遵循这种记法。你应该记住，如果想使用普通矩阵乘法记号，由于 PyTorch 使用 row-major memory ordering，你必须按照公式 1 中行向量约定所示，以转置方式应用矩阵。如果你使用 einsum 进行线性代数操作，只要正确标记轴，这就不应成为问题。顺便一提，Matlab、Julia 和 Fortran 等其他语言/线性代数包都使用 column-major memory ordering，这意味着 batching 维度放在最后；但 Python 及相关包采用了 row-major ordering 的 C 标准。

3.3 基本构建块：Linear 与 Embedding 模块

3.3.1 参数初始化

有效训练神经网络常常需要仔细初始化模型参数——不良初始化可能导致梯度消失或梯度爆炸等不期望的行为。Pre-norm Transformers 对初始化异常鲁棒，但初始化仍然会显著影响训练速度和收敛。由于本作业已经很长，我们会把细节留到作业 3；这里给出一些近似初始化方式，它们在多数情况下应能很好工作。现在使用：

- Linear weights: $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{in}+d_{out}})$，在 $[-3\sigma, 3\sigma]$ 截断。
- Embedding: $\mathcal{N}(\mu = 0, \sigma^2 = 1)$，在 $[-3, 3]$ 截断。
- RMSNorm: **1**。

应使用 `torch.nn.init.trunc_normal_` 初始化截断正态权重。

3.3.2 Linear 模块

Linear layers 是 Transformers 以及一般神经网络中的基本构建块。首先，你将实现自己的 `Linear` 类，它继承自 `torch.nn.Module` 并执行线性变换：

$$y = Wx. \tag{3}$$

注意，遵循大多数现代 LLMs，我们不包含 bias term。

Problem (linear): Implementing the linear module (1 point)

交付内容：实现一个 `Linear` 类，它继承自 `torch.nn.Module` 并执行线性变换。你的实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口，但没有 bias 参数或参数项。我们推荐以下接口：
`def __init__(self, in_features, out_features, device=None, dtype=None)` 构造线性变换模块。此函数应接受以下参数：

`in_features: int`
输入的最终维度

`out_features: int`
输出的最终维度

`device: torch.device | None = None`
存放参数的 device

`dtype: torch.dtype | None = None`
参数的数据类型

def forward(self, x: torch.Tensor) -> torch.Tensor 将线性变换应用于输入。
确保:

- subclass `nn.Module`
- 调用 superclass constructor
- 将参数构造并存储为 W（而不是 $W^\top$），并将其放入 `nn.Parameter`
- 当然，不要使用 `nn.Linear` 或 `nn.functional.linear`

初始化时，使用上面给出的设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。
要测试你的 `Linear` 模块，请实现 `[adapters.run_linear]` 中的 test adapter。adapter 应将给定权重加载到你的 `Linear` 模块中。你可以为此使用 `Module.load_state_dict`。然后运行 `uv run pytest -k test_linear`。

3.3.3 Embedding 模块

如上所述，Transformer 的第一层是一个 embedding 层，它把整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个自定义 `Embedding` 类，它继承自 `torch.nn.Module`（因此不应使用 `nn.Embedding`）。`forward` 方法应使用形状为 `(batch_size, sequence_length)` 的 token ID `torch.LongTensor` 对形状为 `(vocab_size, d_model)` 的 embedding matrix 进行索引，从而为每个 token ID 选择 embedding vector。

Problem (embedding): Implement the embedding module (1 point)

交付内容：实现继承自 `torch.nn.Module` 并执行 embedding lookup 的 `Embedding` 类。你的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们推荐以下接口:
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None) 构造 embedding 模块。此函数应接受以下参数:

num_embeddings: int
词表大小

embedding_dim: int
embedding vectors 的维度，即 d_{model}

device: torch.device | None = None
存放参数的 device

dtype: torch.dtype | None = None
参数的数据类型

def forward(self, token_ids: torch.Tensor) -> torch.Tensor 查找给定 token IDs 对应的 embedding vectors。
确保:

- subclass `nn.Module`
- 调用 superclass constructor
- 将 embedding matrix 初始化为 `nn.Parameter`
- 存储 embedding matrix 时，使 `d_model` 位于最后一维

- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`

再次使用上面给出的设置进行初始化，并使用 `torch.nn.init.trunc_normal_` 初始化权重。要测试你的实现，请实现 `[adapters.run_embedding]` 中的 test adapter。然后运行 `uv run pytest -k test_embedding`。

3.4 Pre-Norm Transformer 块

每个 Transformer block 有两个 sub-layers：一个 multi-head self-attention mechanism 和一个 position-wise feed-forward network（[A. Vaswani et al., 2017]，第 3.1 节）。

在原始 Transformer 论文中，模型在两个 sub-layers 周围各使用一个 residual connection，然后进行 layer normalization。该架构通常称为“post-norm”Transformer，因为 layer normalization 被应用到 sub-layer 输出上。然而，多项工作发现，将 layer normalization 从每个 sub-layer 的输出移动到每个 sub-layer 的输入（并在最终 Transformer block 之后额外加入 layer normalization）可以提高 Transformer 训练稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]——图 2 可视化展示了这种“pre-norm”Transformer block。然后，每个 Transformer block sub-layer 的输出会通过 residual connection 加到 sub-layer 的输入上（A. Vaswani et al. [8]，第 5.4 节）。对于 pre-norm 的一种直觉是，从 input embeddings 到 Transformer 最终输出之间存在一条干净的“residual stream”，其中没有任何 normalization，据称这可以改善 gradient flow。这种 pre-norm Transformer 现在是语言模型中使用的标准架构（例如 GPT-3、LLaMA、PaLM 等），因此我们将实现该变体。我们会依次讲解 pre-norm Transformer block 的每个组件，并逐一实现它们。

3.4.1 均方根层归一化

A. Vaswani et al. [8] 的原始 Transformer 实现使用 layer normalization [J. L. Ba et al., 2016] 来归一化 activations。按照 H. Touvron et al. [12]，我们将使用 root mean square layer normalization（RMSNorm；B. Zhang et al. [13] 公式 4）进行 layer normalization。给定 activations 向量 $a \in \mathbb{R}^{d_{model}}$，RMSNorm 会如下重新缩放每个 activation a_i：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \tag{4}$$

其中 $\text{RMS}(a) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} a_i^2 + \varepsilon}$。这里，$g_i$ 是可学习的“gain”参数（共有 `d_model` 个这样的参数），ε 是一个超参数，通常固定为 `1e-5`。

在对输入取平方之前，你应该将输入 upcast 到 `torch.float32`，以防止 overflow。总体上，你的 `forward` 方法应如下所示：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

交付内容：将 RMSNorm 实现为 `torch.nn.Module`。我们推荐以下接口：
`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)`
构造 RMSNorm 模块。此函数应接受以下参数：

`d_model: int`
模型的 hidden dimension

`eps: float = 1e-5`
数值稳定性所用的 epsilon 值

`device: torch.device | None = None`
存放参数的 device

`dtype: torch.dtype | None = None`
参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量，并返回相同形状的张量。
注意：如上所述，记得在执行归一化前将输入 upcast 到 `torch.float32`（随后再 downcast 到原始 dtype）。
要测试你的实现，请实现 `[adapters.run_rmsnorm]` 中的 test adapter。然后运行 `uv run pytest -k test_rmsnorm`。

3.4.2 按位置前馈网络

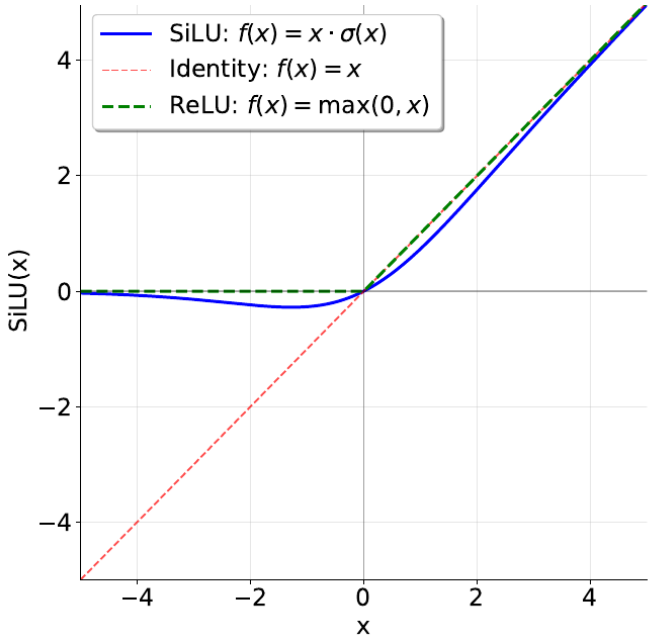


图 3: 比较 SiLU（又名 Swish）与 ReLU 激活函数。

图中文字翻译： SiLU: $f(x) = x \cdot \sigma(x)$ = SiLU：$f(x) = x \cdot \sigma(x)$；Identity: $f(x) = x$ = 恒等函数：$f(x) = x$；ReLU: $f(x) = \max(0, x)$ = ReLU：$f(x) = \max(0, x)$；坐标轴变量 x 和 SiLU(x) 保持原文。

在原始 Transformer 论文（A. Vaswani et al. [8] 第 3.3 节）中，Transformer feed-forward network 由两个线性变换组成，中间使用 ReLU activation（ReLU$(x) = \max(0, x)$）。在原始架构中，内部 feed-forward layer 的维度通常是输入维度的 4x。

然而，与这一原始设计相比，现代语言模型倾向于包含两个主要变化：使用另一种 activation function，并采用 gating mechanism。具体来说，我们将实现 Llama 3 [A. Grattafiori et al., 2024] 和 Qwen 2.5 [A. Yang et al., 2024] 等 LLMs 采用的“SwiGLU” activation function，它将 SiLU（常称为 Swish）activation 与称为 Gated Linear Unit（GLU）的 gating mechanism 结合起来。我们还会省略 linear layers 中有时使用的 bias terms，这遵循 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来大多数现代 LLMs 的做法。

SiLU 或 Swish activation function [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义如下：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \tag{5}$$

从图 3 可见，SiLU activation function 与 ReLU activation function 相似，但在零点处是平滑的。

Gated Linear Units（GLUs）最初由 Y. N. Dauphin et al. [19] 定义为：一个通过 sigmoid function 的线性变换与另一个线性变换的逐元素乘积：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

其中 $\odot$ 表示逐元素乘法。Gated Linear Units 被认为可以“通过为 gradients 提供一条线性路径，同时保留非线性能力，从而减轻深层架构中的 vanishing gradient problem”。

将 SiLU/Swish 与 GLU 结合起来，就得到我们将在 feed-forward networks 中使用的 SwiGLU：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2 \left(\text{SiLU}(W_1 x) \odot W_3 x\right), \tag{7}$$

其中 $x \in \mathbb{R}^{d_{model}}$，$W_1, W_3 \in \mathbb{R}^{d_{ff} \times d_{model}}$，$W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$，并且规范上 $d_{ff} = \frac{8}{3} d_{model}$。在具体实现中，为了硬件效率，将其四舍五入到附近的 64 的倍数是可以的。

N. Shazeer [20] 首先提出将 SiLU/Swish activation 与 GLUs 结合，并通过实验表明 SwiGLU 在语言建模任务上优于 ReLU 和 SiLU（无 gating）等 baseline。稍后在作业中，你将比较 SwiGLU 和 SiLU。虽然我们已经提到这些组件的一些启发式论证（论文中也提供了更多支持证据），但保持经验主义视角很重要：Shazeer 论文中一句如今很有名的话是：

> “我们不解释为什么这些架构似乎有效；与其他一切一样，我们把它们的成功归因于神圣的仁慈。”

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

交付内容：实现 SwiGLU feed-forward network，它由 SiLU activation function 和 GLU 组成。
注意：在这个特定情形中，为了数值稳定性，你可以放心在实现中使用 `torch.sigmoid`。
在你的实现中，应将 d_{ff} 设置为约 $\frac{8}{3} \times d_{model}$，同时确保内部 feed-forward layer 的维度是 64 的倍数，以充分利用硬件。要用我们提供的测试来测试你的实现，需要实现 `[adapters.run_swiglu]` 中的 test adapter。然后运行 `uv run pytest -k test_swiglu` 测试你的实现。

3.4.3 相对位置嵌入

为了向模型注入位置信息，我们将实现 Rotary Position Embeddings [J. Su et al., 2021]，通常称为 RoPE。对于 token 位置 i 处给定的 query token $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$，我们将应用 pairwise rotation matrix R_i，得到 $q'^{(i)} = R_i q^{(i)} = R_i W_q x^{(i)}$。这里，$R_i$ 会以角度 $\theta_{i,k} = \frac{i}{\Theta^{(2k-2)/d}}$ 将 embedding 元素对 $q^{(i)}_{2k-1:2k}$ 作为 2d vectors 旋转，其中 $k \in \{1, \ldots, d/2\}$，Θ 为某个常数。因此，可将 R_i 视为大小为 $d \times d$ 的 block-diagonal matrix，其 blocks 为 R^i_k，$k \in \{1, \ldots, d/2\}$，其中

$$R^i_k = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix}. \tag{8}$$

于是得到完整的旋转矩阵

$$R_i = \begin{pmatrix} R^i_1 & 0 & 0 & \cdots & 0 \\ 0 & R^i_2 & 0 & \cdots & 0 \\ 0 & 0 & R^i_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R^i_{d/2} \end{pmatrix}, \tag{9}$$

其中 0 表示 2×2 zero matrices。虽然可以构造完整的 $d \times d$ 矩阵，但好的解法应利用该矩阵的性质更高效地实现变换。由于我们只关心给定序列中 tokens 的相对旋转，可以在不同层和不同 batch 间复用为 $\cos(\theta_{i,k})$ 与 $\sin(\theta_{i,k})$ 计算得到的值。如果想优化，可以使用一个被所有 layers 引用的 RoPE module，并在 `init` 期间使用 `self.register_buffer(persistent=False)` 创建一个包含 `sin` 和 `cos` 值的 2d 预计算 buffer，而不是 `nn.Parameter`（因为这些固定的 cosine 和 sine 值并不需要学习）。我们对 $q^{(i)}$ 做的完全相同的旋转过程随后也会用于 $k^{(j)}$，按相应的 R_j 旋转。注意，这一层没有可学习参数。

Problem (rope): Implement RoPE (2 points)

交付内容： 实现一个 `RotaryPositionalEmbedding` 类，将 RoPE 应用于输入张量。
推荐以下接口：
`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` 构造 RoPE 模块，并在需要时创建 buffers。

`theta: float`
RoPE 的 Θ 值

`d_k: int`
query 和 key vectors 的维度

`max_seq_len: int`
将输入的最大序列长度

`device: torch.device | None = None`
存放 buffer 的 device

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` 处理形状为 `(..., seq_len, d_k)` 的输入张量，并返回相同形状的张量。注意，你应容忍 x 带有任意数量的 batch 维度。你应假设 token positions 是形状为 `(..., seq_len)` 的张量，用于指定 x 在序列维度上的 token positions。
你应使用 token positions 沿序列维度切片你的（可能预计算的）`cos` 和 `sin` 张量。
要测试你的实现，请完成 `[adapters.run_rope]`，并确保通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力

现在我们将实现 A. Vaswani et al. [8]（第 3.2.1 节）描述的 scaled dot-product attention。作为预备步骤，Attention 操作的定义会使用 softmax，这是一种将未归一化分数向量转换为归一化分布的操作：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

注意，当 v_i 很大时，$\exp(v_i)$ 可能变为 `inf`（于是 $\frac{\text{inf}}{\text{inf}} = \text{NaN}$）。我们可以通过注意到 softmax 操作对所有输入加上任意常数 c 不变来避免这一点。可以利用该性质提升数值稳定性：通常从 v 的所有元素中减去 v 的最大条目，使新的最大条目为 0。现在你将使用这个技巧实现 softmax。

Problem (softmax): Implement softmax (1 point)

交付内容：编写一个函数，对张量应用 softmax 操作。你的函数应接受两个参数：一个张量和一个维度 i，并对输入张量的第 i 个维度应用 softmax。输出张量应与输入张量具有相同形状，但其第 i 个维度现在会包含一个归一化概率分布。使用从第 i 个维度的所有元素中减去该维度最大值的技巧，避免数值稳定性问题。
要测试你的实现，请完成 `[adapters.run_softmax]`，并确保通过 `uv run pytest -k test_softmax_matches_pytorch`。

现在我们可以在数学上定义 Attention 操作如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{11}$$

其中 $Q \in \mathbb{R}^{n \times d_k}$，$K \in \mathbb{R}^{m \times d_k}$，$V \in \mathbb{R}^{m \times d_v}$。这里，$Q$、$K$ 和 V 都是该操作的输入——注意它们不是可学习参数。

Masking(掩码):有时对 attention 操作的输出进行 mask 很方便。mask 应具有形状 $M \in \{\text{True}, \text{False}\}^{n \times m}$，该布尔矩阵的每一行 i 表示 query i 应 attend to 哪些 keys。按照惯例（且略令人困惑），位置 (i, j) 处值为 `True` 表示 query i 会 attend to key j，值为 `False` 表示 query 不会 attend to 该 key。换句话说，值为 `True` 的 (i, j) pairs 处“信息会流动”。例如，考虑一个 entries 为 `[[True, True, False]]` 的 1×3 mask matrix。单个 query vector 只会 attend to 前两个 keys。

从计算角度看，使用 masking 会比在子序列上计算 attention 高效得多；我们可以通过取 pre-softmax 值（$QK^\top/\sqrt{d_k}$）并向 mask matrix 中值为 `False` 的任意条目加上 $-\infty$ 来做到这一点。

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

交付内容：实现 scaled dot-product attention 函数。你的实现应处理形状为 `(batch_size, ..., seq_len, d_k)` 的 keys 和 queries，以及形状为 `(batch_size, ..., seq_len, d_v)` 的 values，其中 `...` 表示任意数量的其他 batch-like 维度（如果提供）。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。关于 batch-like 维度的讨论见第 3.2 节。
你的实现还应支持一个可选的、用户提供的 boolean mask，形状为 `(seq_len, seq_len)`。mask 值为 `True` 的位置的 attention probabilities 应共同和为 1，而 mask 值为 `False` 的位置的 attention probabilities 应为 0。
要用我们提供的测试来测试你的实现，你需要实现 `[adapters.run_scaled_dot_product_attention]` 中的 test adapter。`uv run pytest -k test_scaled_dot_product_attention` 会在三阶输入张量上测试你的实现，而 `uv run pytest -k test_4d_scaled_dot_product_attention` 会在四阶输入张量上测试你的实现。

3.4.5 因果多头自注意力

我们将实现 A. Vaswani et al. [8] 第 3.2.2 节所述的 multi-head self-attention。回忆一下，从数学上，应用 multi-head attention 的操作定义如下：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \tag{12}$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \tag{13}$$

其中 Q_i、K_i、V_i 分别是 Q、K 和 V 的 embedding dimension 中大小为 d_k 或 d_v 的第 $i \in \{1, \ldots, h\}$ 个 slice。Attention 是第 3.4.4 节定义的 scaled dot-product attention 操作。由此可以形成 multi-head self-attention 操作：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \tag{14}$$

这里，可学习参数为 $W_Q \in \mathbb{R}^{hd_k \times d_{model}}$、$W_K \in \mathbb{R}^{hd_k \times d_{model}}$、$W_V \in \mathbb{R}^{hd_v \times d_{model}}$ 和 $W_O \in \mathbb{R}^{d_{model} \times hd_v}$。由于 Q、K 和 V 会在 multi-head attention 操作中被切片，可以认为 W_Q、W_K 和 W_V 沿输出维度为每个 head 分开。当你让这部分工作起来时，应总共用三次 matrix multiplies 计算 key、value 和 query projections。[5]

因果掩码

你的实现应防止模型 attend to 序列中的 future tokens。换句话说，如果模型得到一个 token 序列 $t_1, \ldots, t_n$，并且我们要计算 prefix $t_1, \ldots, t_i$（其中 $i < n$）的下一词预测，模型不应能够访问（attend to）位置 $t_{i+1}, \ldots, t_n$ 的 token representations，因为在推理期间生成文本时它无法访问这些 tokens（而这些 future tokens 会泄露真实下一词身份的信息，使语言建模预训练目标变得 trivial）。对于输入 token 序列 $t_1, \ldots, t_n$，我们可以朴素地通过运行 multi-head self-attention n 次（对应序列中的 n 个 unique prefixes）来防止访问 future tokens。相反，我们将使用 causal attention masking，它允许 token i attend to 序列中的所有位置 $j \leq i$。你可以使用 `torch.triu` 或 broadcasted index comparison 构造这个 mask，并应利用第 3.4.4 节中 scaled dot-product attention 实现已经支持 attention masking 这一事实。

应用 RoPE

RoPE 应应用于 query 和 key vectors，但不应用于 value vectors。此外，head 维度应作为 batch 维度处理，因为在 multi-head attention 中，attention 会对每个 head 独立应用。这意味着应对每个 head 的 query 和 key vectors 应用完全相同的 RoPE rotation。

Problem (multihead_self_attention): Implement causal multi-head self-attention (5 points)

交付内容：将 causal multi-head self-attention 实现为 `torch.nn.Module`。你的实现应至少接受以下参数：

`d_model: int`
Transformer block 输入的维度。

`num_heads: int`
multi-head self-attention 中使用的 heads 数量。

按照 A. Vaswani et al. [8]，设置 $d_k = d_v = \frac{d_{model}}{h}$。要用我们提供的测试来测试你的实现，请实现 `[adapters.run_multihead_self_attention]` 中的 test adapter。然后运行 `uv run pytest -k test_multihead_self_attention` 测试你的实现。

[5] 作为 stretch goal，尝试将 key、query 和 value projections 合并到一个 weight matrix 中，这样只需要一次 matrix multiply。

3.5 完整 Transformer LM

先从组装 Transformer block 开始（回看图 2 会有帮助）。一个 Transformer block 包含两个“sub-layers”，一个用于 multihead self attention，另一个用于 SwiGLU feed-forward network。在每个 sub-layer 中，我们先执行 RMSNorm，然后执行主要操作（MHA/FF），最后加入 residual connection。

具体来说，Transformer block 的前半部分（第一个“sub-layer”）应实现以下更新，用输入 x 生成输出 y：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

Problem (transformer_block): Implement the Transformer block (3 points)

按第 3.4 节描述并如图 2 所示，实现 pre-norm Transformer block。你的 Transformer block 应至少接受以下参数：

`d_model: int`
Transformer block 输入的维度。

`num_heads: int`
multi-head self-attention 中使用的 heads 数量。

`d_ff: int`
position-wise feed-forward 内层的维度。

要测试你的实现，请实现 adapter `[adapters.run_transformer_block]`。然后运行 `uv run pytest -k test_transformer_block` 测试你的实现。
交付内容：通过所提供测试的 Transformer block 代码。

现在我们按照图 1 的高层图示把这些 blocks 组合起来。遵循第 3.1.0.1 节中对 embedding 的描述，将其输入 `num_layers` 个 Transformer blocks，然后传入最终 layer norm 和 LM head，以得到词表上的未归一化分布（logits）。

Problem (transformer_lm): Implementing the Transformer LM (3 points)

是时候把所有部分合在一起了！按第 3.1 节描述并如图 1 所示，实现 Transformer 语言模型。至少，你的实现应接受前述所有 Transformer block 构造参数，并额外接受以下参数：

`vocab_size: int`
词表大小，用于确定 token embedding matrix 的维度。

`context_length: int`
最大上下文长度，用于确定 RoPE `sin` 和 `cos` buffer 的维度。

`num_layers: int`
使用的 Transformer blocks 数量。

要用我们提供的测试来测试你的实现，你首先需要实现 `[adapters.run_transformer_lm]` 中的 test adapter。然后运行 `uv run pytest -k test_transformer_lm` 测试你的实现。
交付内容：一个通过上述测试的 Transformer LM module。

资源核算

理解 Transformer 各部分如何消耗计算和内存是很有用的。我们将逐步进行一些基本的“FLOPs accounting”。Transformer 中绝大多数 FLOPs 来自 matrix multiplies，因此我们的核心方法很简单：

1. 写下 Transformer forward pass 中的所有 matrix multiplies。
2. 将每个 matrix multiply 转换为所需 FLOPs。

第二步会用到以下事实：

规则：给定 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$，matrix-matrix product AB 需要 $2mnp$ FLOPs。

要理解这一点，注意 $(AB)[i, j] = A[i, :] \cdot B[:, j]$，该 dot product 需要 n 次加法和 n 次乘法（$2n$ FLOPs）。然后，由于 matrix-matrix product AB 有 $m \times p$ 个 entries，总 FLOPs 数为 $(2n)(mp) = 2mnp$。

在做下一个问题之前，逐一检查你的 Transformer block 和 Transformer LM 的每个组件，列出所有 matrix multiplies 及其相关 FLOPs costs，会很有帮助。

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

(a) 考虑一个 GPT-2 XL-sized model，使用本作业架构并具有以下配置：

`vocab_size`:
50,257

`context_length`:
1,024

`num_layers`:
48

`d_model`:
1,600

`num_heads`:
25

`d_ff`:
4,288（距离 $\frac{8}{3} \times 1,600$ 最近的 64 的倍数）

假设我们使用该配置构造模型。模型会有多少 trainable parameters？假设每个参数使用 single-precision floating point 表示，仅加载该模型需要多少内存？

交付内容：一到两句回答。

(b) 确定完成我们的 GPT-2 XL-shaped model 的 forward pass 所需的 matrix multiplies。这些 matrix multiplies 总共需要多少 FLOPs？假设输入序列有 `context_length` 个 tokens。

交付内容： matrix multiplies 列表（带描述），以及所需 FLOPs 总数。

(c) 基于上述分析，模型的哪些部分需要最多 FLOPs？

交付内容：一到两句回答。

(d) 使用 GPT-2 small（12 layers, 768 `d_model`, 12 heads）、GPT-2 medium（24 layers, 1024 `d_model`, 16 heads）和 GPT-2 large（36 layers, 1280 `d_model`, 20 heads）重复你的分析。随着模型规模增加，Transformer LM 的哪些部分在总 FLOPs 中占比增大或减小？

交付内容：对每个模型，提供模型组件及其关联 FLOPs 的 breakdown（作为 forward pass 所需总 FLOPs 的比例）。此外，用一到两句话描述改变模型大小如何改变各组件的 FLOPs 占比。

(e) 取 GPT-2 XL 并将 context length 增加到 16,384。一次 forward pass 的总 FLOPs 如何变化？各模型组件对 FLOPs 的相对贡献如何变化？

交付内容：一到两句回答。

4 训练 Transformer LM

我们现在已经有了预处理数据（通过分词器）和模型（Transformer）的步骤。剩下的是构建支持训练的所有代码。这包括：

- **损失：**需要定义损失函数（cross-entropy）。
- **优化器：**需要定义用于最小化该损失的优化器（AdamW）。
- **训练循环：**需要所有支持基础设施，用于加载数据、保存 checkpoints 和管理训练。

4.1 交叉熵损失

回忆一下，对于长度为 $m+1$ 的每个序列 x 和 $i = 1, \ldots, m$，Transformer 语言模型定义分布 $p_\theta(x_{i+1} \mid x_{1:i})$。给定由长度为 $m+1$ 的序列组成的训练集 D，我们定义标准 cross-entropy（negative log-likelihood）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

（注意，Transformer 中的一次 forward pass 会为所有 $i = 1, \ldots, m$ 产生 $p_\theta(x_{i+1} \mid x_{1:i})$。）

具体来说，Transformer 为每个位置 i 计算 logits $o_i \in \mathbb{R}^{\text{vocab_size}}$，从而得到：[6]

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \tag{17}$$

Cross-entropy loss 通常相对于 logits 向量 $o_i \in \mathbb{R}^{\text{vocab_size}}$ 和目标 x_{i+1} 定义。[7]

实现 cross-entropy loss 与 softmax 情况一样，需要注意一些数值问题。

Problem (cross_entropy): Implement cross-entropy (1 point)

交付内容：编写一个函数来计算 cross-entropy loss，该函数接收预测 logits（o_i）和 targets（x_{i+1}），并计算 cross-entropy $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$。你的函数应处理以下内容：

- 为数值稳定性减去最大元素。
- 尽可能抵消 `log` 和 `exp`。
- 处理任何额外 batch 维度并返回整个 batch 的平均值。与第 3.2 节一样，我们假设 batch-like 维度总是在前面，位于 vocabulary size 维度之前。

实现 `[adapters.run_cross_entropy]`，然后运行 `uv run pytest -k test_cross_entropy` 测试你的实现。

[6] 注意，$o_i[k]$ 表示向量 o_i 中索引 k 处的值。

[7] 这对应于 x_{i+1} 上的 Dirac delta distribution 与预测的 softmax(o_i) 分布之间的 cross-entropy。

困惑度

Cross-entropy 足以用于训练，但在评估模型时，我们还希望报告 perplexity。对于长度为 m 的序列，如果承受 cross-entropy losses $\ell_1, \ldots, \ell_m$：

$$\text{perplexity} = \exp\left(\frac{1}{m}\sum_{i=1}^{m}\ell_i\right). \tag{18}$$

4.2 SGD 优化器

现在有了损失函数，我们将开始探索优化器。最简单的基于梯度的优化器是 Stochastic Gradient Descent（SGD）。我们从随机初始化的参数 θ_0 开始。然后，对于每一步 $t = 0, \ldots, T-1$，执行以下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

其中 B_t 是从数据集 D 中采样的随机 batch，learning rate α_t 和 batch size $|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现我们的优化器，我们将 subclass PyTorch 的 `torch.optim.Optimizer` 类。一个 `Optimizer` subclass 必须实现两个方法：

`def __init__(self, params, ...)` 应初始化优化器。这里，`params` 是要优化的参数集合（或 parameter groups，以便用户希望对模型不同部分使用不同超参数，例如不同 learning rates）。确保将 `params` 传给基类的 `__init__` 方法，基类会保存这些参数供 `step` 使用。你可以根据优化器接受额外参数（例如 learning rate 很常见），并以字典形式将它们传给基类构造函数，字典键是你为这些参数选择的名称（字符串）。

`def step(self)` 应对参数做一次更新。在训练循环中，它会在 backward pass 之后被调用，因此你可以访问上一 batch 上的 gradients。此方法应遍历每个参数张量 `p`，并原地修改它们，即设置 `p.data`，它保存与该参数相关的张量，修改依据是 `p.grad`（如果存在），即表示损失相对于该参数的 gradient 的张量。

PyTorch 优化器 API 有一些微妙之处，因此用例子解释更容易。为了让例子更丰富，我们将实现一种 SGD 的轻微变体，其中 learning rate 会随着训练衰减，从初始 learning rate α 开始，并随时间采用逐渐变小的步长：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}}\nabla L(\theta_t; B_t). \tag{20}$$

下面看看这个版本的 SGD 如何实现为 PyTorch `Optimizer`：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)
```

```
    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # Get state associated with p.
                t = state.get("t", 0) # Get iteration number from the state, or 0.
                grad = p.grad.data # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad # Update weight tensor in-place.
                state["t"] = t + 1 # Increment iteration number.

        return loss
```

在 `__init__` 中，我们将参数以及默认超参数传给基类构造函数（这些参数可能分组，每组有不同超参数）。如果参数只是单个 `torch.nn.Parameter` 对象集合，基类构造函数会创建单个 group，并为其分配默认超参数。然后，在 `step` 中，我们遍历每个 parameter group，再遍历该 group 中的每个参数，并应用公式 20。这里，我们将迭代次数保存为与每个参数相关联的 state：先读取该值，在 gradient update 中使用它，然后更新它。API 规定用户可以传入一个 callable `closure`，用于在优化器步骤前重新计算 loss。我们将使用的优化器不需要这一点，但为了符合 API，会添加它。

要看到它运行，可以使用下面这个最小 training loop 示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
    loss.backward() # Run backward pass, which computes gradients.
    opt.step() # Run optimizer step.
```

这是 training loop 的典型结构：在每次迭代中，计算 loss 并运行优化器的一步。训练语言模型时，可学习参数来自模型（在 PyTorch 中，`m.parameters()` 会给出这个集合）。loss 会在采样的 data batch 上计算，但训练循环的基本结构相同。

Problem (learning_rate_tuning): Tuning the learning rate (1 point)

如我们将看到的，learning rate 是最影响训练的超参数之一。让我们在 toy example 中实际观察。用另外三个 learning rate 值运行上面的 SGD 示例：`1e1`、`1e2` 和 `1e3`，只运行 10 次 training iterations。对于每个 learning rate，loss 会发生什么？它衰减得更快、更慢，还是会发散（即在训练过程中增加）？
交付内容：一到两句回答，说明你观察到的行为。

4.3 AdamW

现代语言模型通常使用比 SGD 更复杂的优化器训练。最近使用的大多数优化器都是 Adam 优化器 [D. P. Kingma et al., 2015] 的衍生版本。我们将使用 AdamW [I. Loshchilov et al., 2019]，它在近期工

作中被广泛使用。AdamW 对 Adam 提出了一种修改：通过加入 weight decay（每次迭代将参数拉向 0）改善 regularization，并且这种方式与 gradient update 解耦。我们将按照 I. Loshchilov et al. [23] Algorithm 2 中的描述实现 AdamW。

AdamW 是有状态的：对于每个参数，它会跟踪其一阶和二阶 moments 的 running estimate。因此，AdamW 使用额外内存来换取更好的稳定性和收敛性。除 learning rate α 外，AdamW 还有一对超参数 (β_1, β_2)，用于控制 moment estimates 的更新，以及一个 weight decay rate λ。典型应用会将 (β_1, β_2) 设为 $(0.9, 0.999)$，但 LLaMA [H. Touvron et al., 2023] 和 GPT-3 [T. B. Brown et al., 2020] 等大型语言模型常用 $(0.9, 0.95)$。该算法可写作如下，其中 ε 是一个很小的值（例如 10^{-8}），用于在 v 中出现极小值时改善数值稳定性：

算法 1：AdamW 优化器

init(θ) ▷ 初始化可学习参数
$m \leftarrow 0$ ▷ 一阶 moment vector 的初始值；与 θ 形状相同
$v \leftarrow 0$ ▷ 二阶 moment vector 的初始值；与 θ 形状相同
for $t = 1, \ldots, T$ **do**
  采样一批数据 B_t
  $g \leftarrow \nabla_\theta \ell(\theta; B_t)$ ▷ 计算 loss 的 gradient
  $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$ ▷ 计算 iteration t 的调整后 α
  $\theta \leftarrow \theta - \alpha\lambda\theta$ ▷ 应用 weight decay
  $m \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ 更新一阶 moment estimate
  $v \leftarrow \beta_2 v + (1-\beta_2) g^2$ ▷ 更新二阶 moment estimate
  $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\varepsilon}$ ▷ 应用 moment-adjusted weight updates
end for

Algorithm 1: AdamW optimizer pseudocode.

注意 t 从 1 开始。现在你将实现这个优化器。

Problem (adamw): Implement AdamW (2 points)

交付内容： 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的 subclass。你的类应在 `__init__` 中接收 learning rate α，以及 β、ε 和 λ 超参数。为了帮助你保存 state，基类 `Optimizer` 提供了一个字典 `self.state`，它将 `nn.Parameter` 对象映射到一个字典，该字典可存储你需要为该参数保存的任何信息（对 AdamW 来说，即 moment estimates）。实现 `[adapters.get_adamw_cls]`，并确保它通过 `uv run pytest -k test_adamw`。

Problem (adamw_accounting): Resource accounting for training with AdamW (2 points)

让我们计算运行 AdamW 需要多少内存和计算。假设每个张量都使用 `float32`。

(a) 运行 AdamW 需要多少 peak memory？请基于 parameters、activations、gradients 和 optimizer state 的内存使用分解你的答案。以 `batch_size` 和模型超参数（`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`）表达你的答案。假设 $d_{ff} = \frac{8}{3} \times d_{model}$。

为简化起见，在计算 activations 的内存使用时，只考虑以下组件：

- Transformer block
  - RMSNorm(s)
  - Multi-head self-attention sublayer: QKV projections、$QK^\top$ matrix multiply、softmax、values 的 weighted sum、output projection。
  - Position-wise feed-forward（SwiGLU）：W_1、W_2、gate branch 上的 SiLU、逐元素乘积、W_3。
- final RMSNorm
- output embedding
- logits 上的 cross-entropy

交付内容： parameters、activations、gradients 和 optimizer state 的代数表达式，以及总和。

(b) 将你的答案实例化为 GPT-2 XL-shaped model，得到只依赖于 `batch_size` 的表达式。要仍然适配 80GB memory，最大 batch size 是多少？

交付内容： 形如 $a \cdot \text{batch_size} + b$ 的表达式，其中 a, b 为数值，以及表示最大 batch size 的数。

(c) 运行一步 AdamW 需要多少 FLOPs？

交付内容： 一个代数表达式，并附简要说明。

(d) Model FLOPs utilization（MFU）定义为 observed throughput（tokens per second）相对于硬件 theoretical peak FLOP throughput 的比率 [A. Chowdhery et al., 2022]。NVIDIA H100 GPU 对“float32”（实际上是 TensorFloat-32，现实中是“bfloat19”）操作的 theoretical peak 为 495 teraFLOP/s。假设你能达到 50% MFU，在单个 H100 上以 batch size 1024 训练 GPT-2 XL 400K steps 需要多久？按照 J. Kaplan et al. [25] 和 J. Hoffmann et al. [26]，假设 backward pass 的 FLOPs 是 forward pass 的两倍。

交付内容： 训练需要的小时数，并附简要说明。

4.4 学习率调度

导致 loss 最快下降的 learning rate 数值在训练期间往往会变化。训练 Transformers 时，典型做法是使用 learning rate schedule：开始时使用较大的 learning rate，使早期更新更快，然后随着模型训练逐渐衰减到较小值。[8] 在本作业中，我们将实现用于训练 LLaMA [H. Touvron et al., 2023] 的 cosine annealing schedule。

Scheduler 只是一个函数：它接收当前 step t 和其他相关参数（例如初始和最终 learning rates），并返回 step t 的 gradient update 应使用的 learning rate。最简单的 schedule 是常数函数，它对任意 t 返回相同的 learning rate。

Cosine annealing learning rate schedule 接收 (i) 当前 iteration t，(ii) 最大 learning rate α_{max}，(iii) 最小（最终）learning rate α_{min}，(iv) warm-up iterations 数 T_w，以及 (v) cosine annealing 的最终 iteration T_c。iteration t 的 learning rate 定义如下：

（预热）如果 $t < T_w$，则 $\alpha_t = \frac{t}{T_w}\alpha_{max}$。

[8] 有时也常用一种 learning rate 重新升高（restarts）的 schedule，以帮助越过 local minima。

（余弦退火）如果 $T_w \le t \le T_c$，则

$$\alpha_t = \alpha_{min} + \frac{1}{2}\left(1 + \cos\left(\frac{t - T_w}{T_c - T_w}\pi\right)\right)(\alpha_{max} - \alpha_{min}).$$

（退火后）如果 $t > T_c$，则 $\alpha_t = \alpha_{min}$。

Problem (learning_rate_schedule): Implement cosine learning rate schedule with warmup (1 point)

编写一个函数，接收 t、α_{max}、α_{min}、T_w 和 T_c，并按上述 scheduler 返回 learning rate α_t。然后实现 `[adapters.get_lr_cosine_schedule]`，并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 梯度裁剪

训练期间，有时会遇到产生大 gradients 的训练样本，这可能使训练不稳定。为缓解这一问题，实践中常用的一种技术是 gradient clipping。其思想是在每次 backward pass 之后、执行 optimizer step 之前，对 gradient 的 norm 施加上限。

给定（所有参数的）gradient g，我们计算其 ℓ_2-norm $\|g\|_2$。如果该 norm 小于最大值 M，则保持 g 不变；否则，用因子 $\frac{M}{\|g\|_2+\varepsilon}$ 缩小 g（其中加入一个很小的 ε，例如 10^{-6}，用于数值稳定性）。注意，得到的 norm 会略小于 M。

Problem (gradient_clipping): Implement gradient clipping (1 point)

编写一个实现 gradient clipping 的函数。你的函数应接收一个参数列表和一个最大 ℓ_2-norm。它应原地修改每个参数的 gradient。使用 $\varepsilon = 10^{-6}$（PyTorch 默认值）。然后实现 adapter `[adapters.run_gradient_clipping]`，并确保它通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环

现在，我们终于要把迄今构建的主要组件组合起来：tokenized data、模型和优化器。

5.1 数据加载器

Tokenized data（例如你在 `tokenizer_experiments` 中准备的数据）是一个单一 token 序列 $x = (x_1, \ldots, x_n)$。即使源数据可能由多个独立文档组成（例如不同网页或源代码文件），一种常见做法是将它们全部连接成一个 token 序列，并在其间添加 delimiter（例如 `<|endoftext|>` token）。

Data loader 将其转换为 batch 流，其中每个 batch 由 B 个长度为 m 的序列组成，并配有对应的 next tokens，长度同样为 m。例如，对于 $B = 1$、$m = 3$，$([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就是一个可能的 batch。

以这种方式加载数据会从多方面简化训练。首先，任意 $1 \le i \le n - m$ 都给出有效训练序列，因此采样训练序列很直接。由于所有训练序列长度相同，无需 pad input sequences，这能提高硬件利用率（也能通过增大 batch size B 提高利用率）。最后，我们也不需要加载完整数据集来采样训练数据，因此可以轻松处理本来可能无法放入内存的大型数据集。

Problem (data_loading): Implement data loading (2 points)

交付内容：编写一个函数，接收一个 numpy array x（含 token IDs 的整数数组）、一个 `batch_size`、一个 `context_length` 和一个 PyTorch device string（例如 `'cpu'` 或 `'cuda:0'`），并返回一对张量：采样的输入序列和对应的 next-token targets。两个张量都应具有形状 `(batch_size, context_length)`，包含 token IDs，并且都放置在请求的 device 上。要用我们提供的测试来测试你的实现，你首先需要实现 `[adapters.run_get_batch]` 中的 test adapter。然后运行 `uv run pytest -k test_get_batch` 测试你的实现。

Low-Resource Tip: Data loading on CPU or Apple Silicon

如果你计划在 CPU 或 Apple Silicon 上训练 LM，需要把数据移动到正确的 device（类似地，之后也应对模型使用同一 device）。
如果使用 CPU，可以使用 `'cpu'` device string；在 Apple Silicon（M* chips）上，可以使用 `'mps'` device string。
关于 MPS 的更多信息，请查看这些资源：

- https://docs.pytorch.org/docs/stable/mps.html
- https://docs.pytorch.org/docs/stable/notes/mps.html
- https://developer.apple.com/documentation/metalperformanceshaders

如果数据集太大，无法加载到内存中怎么办？我们可以使用名为 `mmap` 的 Unix system call，它会将磁盘上的文件映射到 virtual memory，并在访问该内存位置时惰性加载文件内容。因此，你可以“假装”已经把整个数据集放入内存。NumPy 通过 `np.memmap` 实现这一点（如果你最初使用 `np.save` 保存数组，也可使用 `np.load` 的 `mmap_mode='r'` 标志），它会返回一个类似 numpy array 的对象，在你访问 entries 时按需加载。在训练期间从数据集（即 numpy array）中采样时，请务必以 memory-mapped mode 加载数据集（通过 `np.memmap` 或 `np.load` 的 `mmap_mode='r'` 标志，取决于你的数组保存方式）。确保还指定与所加载数组匹配的 `dtype`。显式验证 memory-mapped data 看起来正确可能很有帮助（例如，不包含超出预期词表大小的值）。

5.2 检查点

除了加载数据，我们还需要在训练时保存模型。运行 jobs 时，我们经常希望能够恢复中途停止的训练运行（例如由于 job 超时、机器故障等）。即使一切顺利，我们也可能希望之后访问 intermediate models（例如用于事后研究 training dynamics、从不同训练阶段的模型采样等）。

Checkpoint 应包含恢复训练所需的所有 states。至少，我们当然希望能够恢复 model weights。如果使用有状态优化器（例如 AdamW），还需要保存 optimizer state（例如 AdamW 中的 moment estimates）。最后，为了恢复 learning rate schedule，我们需要知道停止时的 iteration number。PyTorch 使保存这些内容变得容易：每个 `nn.Module` 都有 `state_dict()` 方法，该方法返回一个包含所有 learnable weights 的字典；之后可以用 sister method `load_state_dict()` 恢复这些 weights。任何 `torch.optim.Optimizer` 也是如此。最后，`torch.save(obj, dest)` 可以将一个 object（例如一个字典，其中一些值为 tensors，也可以包含像 integers 这样的普通 Python objects）dump 到文件（路径）或 file-like object 中，之后可以用 `torch.load(src)` 加载回内存。

Problem (checkpointing): Implement model checkpointing (1 point)

实现以下两个函数，用于加载和保存 checkpoints：

`def save_checkpoint(model, optimizer, iteration, out)` 应将 model、optimizer 和 iteration 的全部 state dump 到 file-like object `out`。可以使用 model 和 optimizer 的 `state_dict` 方法获取相关 states，并用 `torch.save(obj, out)` 将 `obj` dump 到 `out` 中（PyTorch 在这里支持 path 或 file-like object）。典型选择是让 `obj` 成为字典，但只要之后能加载你的 checkpoint，可以使用任何格式。

此函数期望以下参数：

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`
- `iteration: int`
- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`def load_checkpoint(src, model, optimizer)` 应从 `src`（path 或 file-like object）加载 checkpoint，然后从该 checkpoint 恢复 model 和 optimizer states。你的函数应返回保存到 checkpoint 中的 iteration number。可以使用 `torch.load(src)` 恢复你在 `save_checkpoint` 实现中保存的内容，并用 model 和 optimizer 的 `load_state_dict` 方法将它们恢复到先前 state。

此函数期望以下参数：

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]` adapters，并确保它们通过 `uv run pytest -k test_checkpointing`。

5.3 训练循环

现在终于到了把你实现的所有组件放入主训练脚本的时候。让启动不同超参数的训练运行变得容易会很有回报（例如通过命令行参数传入它们），因为之后你会多次这样做来研究不同选择如何影响训练。

Problem (training_together): Put it together (4 points)

交付内容：编写一个脚本，运行 training loop，在用户提供的输入上训练你的模型。特别地，我们建议你的训练脚本至少允许以下内容：

- 能够配置和控制各种模型与优化器超参数。
- 使用 `np.memmap` 对大型训练集与验证集进行内存高效加载。
- 将 checkpoints 序列化到用户提供的路径。
- 周期性地记录 training 和 validation performance（例如记录到 console 和/或 Weights and Biases 等外部服务）。[a]

[a] https://wandb.ai

6 生成文本

现在我们可以训练模型了，最后一块是让模型能够生成文本。回忆一下，语言模型接收长度为 `sequence_length` 的（可能是 batched 的）整数序列，并生成大小为 `(sequence_length, vocab_size)` 的矩阵，其中序列的每个元素都是一个概率分布，用于预测该位置之后的下一个 token。现在我们将编写几个函数，把它转化为新序列的采样方案。

Softmax

按照标准约定，语言模型输出是最后一个 linear layer 的输出（"logits"），因此我们必须通过 softmax 操作将其转换为归一化概率；这一操作我们已在公式 10 中见过。

解码

为了从模型生成文本（decode），我们会向模型提供一串 prefix tokens（"prompt"），并要求它生成词表上的概率分布，用来预测序列中的下一个 token。然后，我们会从这个词表项分布中采样，以确定下一个输出 token。

具体来说，decoding process 的一步应接收序列 $x_{1\ldots t}$，并通过以下方程返回一个 token x_{t+1}：

$$P(x_{t+1} = i \mid x_{1\ldots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \tag{21}$$

$$v = \text{TransformerLM}(x_{1\ldots t})_t \in \mathbb{R}^{\text{vocab_size}} \tag{22}$$

其中 `TransformerLM` 是我们的模型，它接收长度为 `sequence_length` 的序列作为输入，并生成大小为 `(sequence_length, vocab_size)` 的矩阵；我们取该矩阵的最后一个元素，因为我们要找的是第 t 个位置的 next token prediction。

这给出了一个基本 decoder：反复从这些 one-step conditionals 中采样（将前一时刻生成的输出 token 追加到下一 decoding timestep 的输入中），直到生成 end-of-sequence token `<|endoftext|>`（或生成达到用户指定的最大 token 数）。

解码技巧

我们将用小模型做实验，而小模型有时会生成质量很低的文本。两个简单 decoder tricks 可以帮助修正这些问题。首先，在 temperature scaling 中，我们使用 temperature parameter τ 修改 softmax，其中新的 softmax 为

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \tag{23}$$

注意，令 $\tau \to 0$ 会使 v 的最大元素占主导，softmax 输出变为集中在该最大元素上的 one-hot vector。

第二，另一种技巧是 nucleus 或 top-p sampling，即通过截断低概率 tokens 修改采样分布。令 q 为我们从大小为 `vocab_size` 的（temperature-scaled）softmax 得到的概率分布。带超参数 p 的 nucleus sampling 按下式生成下一个 token：

$$P(x_{t+1} = i \mid q) = \begin{cases} \dfrac{q_i}{\sum_{j \in V(p)} q_j}, & \text{if } i \in V(p), \\ 0, & \text{otherwise}, \end{cases} \tag{24}$$

其中 $V(p)$ 是满足 $\sum_{j \in V(p)} q_j \geq p$ 的最小索引集合。你可以先按大小排序概率分布 q，再选择最大的词表元素，直到达到目标 level p，即可轻松计算该量。

Problem (decoding): Decoding (3 points)

交付内容：实现一个从语言模型 decode 的函数。我们建议你支持以下功能：

- 为用户提供的 prompt 生成 completions（即接收某些 $x_{1...t}$，并采样 completion，直到遇到 `<|endoftext|>` token）。
- 允许用户控制最大生成 token 数。
- 给定所需 temperature value，在采样前对预测 next-token distributions 应用 softmax temperature scaling。
- 给定用户指定的 threshold value，进行 Top-p sampling（[A. Holtzman et al., 2020]，也称为 nucleus sampling）。

7 实验

现在是时候把所有内容组合起来，并在预训练数据集上训练（小型）语言模型了。

7.1 如何运行实验与交付内容

理解 Transformer 架构组件背后理由的最佳方式，是亲自修改它并运行。没有什么可以替代 hands-on experience。

为此，能够快速、一致地实验并记录你做过什么非常重要。为了快速实验，我们会在小规模模型（总参数约 17M）和简单数据集（TinyStories）上运行许多实验。为了保持一致，你将系统性地消融组件并改变超参数；为了保留记录，我们会要求你提交实验日志以及每个实验对应的 learning curves。

为了能够提交 loss curves，请确保周期性评估 validation losses，并记录 steps 数和 wall-clock times。你可能会发现 Weights and Biases 等 logging infrastructure 很有帮助。

Problem (experiment_log): Experiment logging (3 points)

对于你的 training 和 evaluation 代码，创建 experiment tracking infrastructure，使你能够按 gradient steps 和 wall-clock time 跟踪实验和 loss curves。
交付内容：用于实验的 logging infrastructure 代码，以及本节下面作业问题的 experiment log（记录你尝试过的所有事项的文档）。

7.2 TinyStories

我们将从一个非常简单的数据集（TinyStories；R. Eldan et al. [1]）开始，在这个数据集上模型训练很快，并且可以看到一些有趣的行为。获取该数据集的说明见第 1 节。下面是该数据集中一个示例。

示例 (tinystories_example)：TinyStories 中的一个示例

从前有一个名叫 Ben 的小男孩。Ben 喜欢探索他周围的世界。他看到了许多令人惊叹的东西，比如商店里陈列的漂亮花瓶。一天，Ben 走过商店时，看到一个非常特别的花瓶。当 Ben 看到它时，他惊叹不已！他说：“哇，那真是一个非常惊人的花瓶！我可以买它吗?”店主微笑着

说："当然可以。你可以把它带回家，给你所有的朋友看它有多么惊人！"于是 Ben 把花瓶带回了家，他为此感到非常自豪！他把朋友们叫过来，给他们看那个惊人的花瓶。他所有朋友都觉得花瓶很漂亮，也不敢相信 Ben 这么幸运。Ben 就这样在商店里找到了一个令人惊叹的花瓶！

7.2.1 超参数调优

我们会告诉你一些非常基本的起始超参数，并要求你找到其他效果良好的设置。

词表大小 10000。典型词表大小在数万到数十万之间。你应该改变该值，观察词表和模型行为如何变化。

上下文长度 256。 TinyStories 这样的简单数据集可能不需要很长的序列长度，但对于后面的 OpenWebText 数据，你可能想改变它。尝试改变该值，并观察它对每次迭代运行时间和最终困惑度的影响。

d_model 512. 这略小于许多小型 Transformer 论文使用的 768 维，但会更快。

d_ff 1344. 这大约是 $\frac{8}{3}d_{model}$，同时是 64 的倍数，有利于 GPU 性能。

RoPE theta 参数 Θ 10000。

层数和 heads 数：4 层，16 heads。合起来这会得到约 17M 非 embedding 参数，是一个相当小的 Transformer。

处理的总 tokens 数 327,680,000（你的 batch size $\times$ total step count $\times$ context length 应大致等于这个值）。

你应通过一些 trial and error, 为以下其他超参数找到良好的默认值：learning rate、learning rate warmup、其他 AdamW 超参数（$\beta_1, \beta_2, \varepsilon$）以及 weight decay。你可以在 D. P. Kingma et al. [22] 中找到这些超参数的一些典型选择。

7.2.2 组合起来

现在你可以通过获取训练好的 BPE 分词器、对训练数据集进行分词，并在你编写的 training loop 中运行它，把所有内容组合起来。重要说明：如果你的实现正确且高效，上述超参数在 1 块 B200 GPU 上应产生约 20–30 分钟的运行时间。如果你的运行时间长得多，请检查并确保 dataloading、checkpointing 或 validation loss code 没有成为运行时瓶颈，并且你的实现进行了正确的 batching。

7.2.3 调试模型架构的提示与技巧

我们强烈建议你熟悉 IDE 内置的 debugger（例如 VSCode/Zed），这会比用 print statements 调试节省时间。如果你使用文本编辑器，可以使用类似 `ipdb` 的工具。调试模型架构时，还有一些其他好习惯：

- 开发任何神经网络架构时，一个常见的第一步是在单个 minibatch 上 overfit。如果实现正确，应能很快把 training loss 降到接近零。
- 在各种模型组件中设置 debug breakpoints，并检查 intermediate tensors 的 shapes，确保它们符合你的预期。
- 监控 activations、model weights 和 gradients 的 norms，确保它们没有 exploding 或 vanishing。

Problem (learning_rate): Tune the learning rate (2 B200 hrs) (3 points)

Learning rate 是最重要的调优超参数之一。基于你训练好的 base model，回答以下问题：

(a) 对 learning rates 进行 hyperparameter sweep，并报告最终 losses（如果 optimizer 发散，则注明 divergence）。

交付内容：与多个 learning rates 相关的 learning curves。解释你的 hyperparameter search strategy。

交付内容：一个在 TinyStories 上 validation loss（per-token）至多为 1.45 的模型。

Low-Resource Tip: Train for a few steps on CPU or Apple Silicon

如果你在 `cpu` 或 `mps` 上运行，应将 total tokens processed 数减少到 40,000,000，这足以产生相当流畅的文本。你也可以将目标 validation loss 从 1.45 提高到 2.00。
在 M4 Max 芯片和 36 GB RAM 上运行我们的 solution code，并使用调优后的 learning rate 时，我们使用 batch size × total step count × context length $= 32 \times 5000 \times 256 = 40,960,000$ tokens，在 `cpu` 上耗时 1 小时 22 分钟，在 `mps` 上耗时 36 分钟。在 step 5000，我们达到 1.80 的 validation loss。
一些额外提示：

- 使用 N 个 training steps 时，我们建议调整 cosine learning rate decay schedule，使其 decay 在 precisely step N 处终止（即达到 minimum learning rate）。
- 使用 `mps` 时，不要使用 TF32 kernels，即不要像在 `cuda` devices 上可能会做的那样设置：

  ```
  torch.set_float32_matmul_precision('high')
  ```

  我们尝试在 `mps`（`torch` version 2.9.0）上启用 TF32 kernels，发现 backend 有时会使用 silently broken kernels，导致训练不稳定。
- 可以通过用 `torch.compile` JIT 编译模型来加速训练。具体来说：
  - 在 `cpu` 上，用以下方式编译模型：

    ```
    model = torch.compile(model)
    ```
  - 在 `mps` 上，可以用以下方式稍微优化 backward pass：

    ```
    model = torch.compile(model, backend="aot_eager")
    ```

  截至 `torch` version 2.9.0，Inductor compilation 在 `mps` 上不受支持。

(b) 民间经验认为，最好的 learning rate 位于“edge of stability”。研究 learning rates 发散的点与你的最佳 learning rate 之间的关系。

交付内容：递增 learning rate 的 learning curves，其中至少包括一次 divergent run，并分析其与 convergence rates 的关系。

现在让我们改变 batch size，看看训练会发生什么。Batch sizes 很重要——它们让我们通过更大的 matrix multiplies 从 GPU 获得更高效率，但我们是否总是希望 batch sizes 很大？让我们运行一些实验来弄清楚。

Problem (batch_size_experiment): Batch size variations (1 B200 hr) (1 point)

从 1 一直到 GPU memory limit 改变你的 batch size。在中间至少尝试几个 batch sizes，包括 64 和 128 等典型大小。
交付内容：不同 batch sizes 运行的 learning curves。必要时应重新优化 learning rates。
交付内容：几句话讨论你关于 batch sizes 及其对训练影响的发现。

有了 decoder 后，我们现在可以生成文本了！我们将从模型生成文本并观察其质量。作为参考，你应该得到至少和下面示例一样好的输出。

示例 (ts_generate_example)：TinyStories 语言模型的样本输出

从前，有一个名叫 Lily 的漂亮女孩。她喜欢吃口香糖，尤其是那个大的黑色口香糖。一天，Lily 的妈妈让她帮忙做晚饭。Lily 非常兴奋！她喜欢帮助妈妈。Lily 的妈妈为晚餐做了一大锅汤。Lily 非常高兴，说："谢谢你，妈妈！我爱你。"她帮妈妈把汤倒进一个大碗里。晚饭后，Lily 的妈妈又做了一些美味的汤。Lily 很喜欢！她说："谢谢你，妈妈！这汤真好喝！"她的妈妈微笑着说："我很高兴你喜欢它，Lily。"她们做完饭后继续一起烹饪。结束。

Low-Resource Tip: Generate text on CPU or Apple Silicon

如果你使用的是处理 40M tokens 的低资源配置，你应该看到生成内容仍然像英语，但不如上面的示例流畅。例如，我们从一个在 40M tokens 上训练的 TinyStories 语言模型得到的 sample output 如下：
从前，有一个名叫 Sue 的小女孩。Sue 有一颗她非常喜欢的牙齿。它是他最好的头。一天，Sue 出去散步，遇见了一只瓢虫！她们成了好朋友，并一起在小路上玩耍。
"嘿，Polly！我们出去吧！"Tim 说。Sue 看着天空，发现很难找到一种跳着发光舞蹈的方式。她微笑着同意帮助那段说话！
当 Sue 看着天空移动时，那是什么。她

下面是精确的问题陈述以及我们要求你完成的内容：

Problem (generate): Generate text (1 point)

使用你的 decoder 和训练好的 checkpoint，报告你的模型生成的文本。你可能需要调整 decoder 参数（temperature、top-p 等）来获得流畅输出。
交付内容：至少 256 个 tokens 的文本 dump（或直到第一个 `<|endoftext|>` token），并简要评论该输出的流畅性，以及至少两个影响输出好坏的因素。

7.3 消融与架构修改

理解 Transformer 的最佳方式是实际修改它并观察其行为。现在我们会做几个简单的 ablations 和 modifications。

消融 1：layer normalization

常有人说 layer normalization 对 Transformer 训练稳定性很重要。但也许我们想大胆冒险。让我们从每个 Transformer block 中移除 RMSNorm，看看会发生什么。

Problem (layer_norm_ablation): Remove RMSNorm and train (0.5 B200 hrs) (1 point)

从你的 Transformer 中移除所有 RMSNorms 并训练。在之前的最佳 learning rate 下会发生什么？能否通过使用更低的 learning rate 获得稳定性？
交付内容：移除 RMSNorms 后训练的 learning curve，以及最佳 learning rate 的 learning curve。
交付内容：几句话评论 RMSNorm 的影响。

现在让我们研究另一个乍看有些任意的 layer normalization 选择。Pre-norm Transformer blocks 定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \tag{25}$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \tag{26}$$

这是对原始 Transformer 架构少数几个“共识性”修改之一；原始架构使用 post-norm 方法：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \tag{27}$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \tag{28}$$

让我们回到 post-norm 方法，看看会发生什么。

Problem (pre_norm_ablation): Implement post-norm and train (0.5 B200 hrs) (1 point)

将你的 pre-norm Transformer 实现修改为 post-norm。用 post-norm model 训练并观察会发生什么。
交付内容： post-norm Transformer 与 pre-norm Transformer 的 learning curve 对比。

我们看到，layer normalization 对 Transformer 行为有重大影响，甚至 layer normalization 的位置也很重要。

消融 2：position embeddings

接下来我们将研究 position embeddings 对模型性能的影响。具体来说，我们会比较 base model（使用 RoPE）和完全不包含 position embeddings 的模型（NoPE）。事实证明，decoder-only transformers，也就是我们实现的带 causal mask 的 transformers，理论上可以在没有显式提供 position embeddings 的情况下推断相对或绝对位置信息 [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。现在我们将经验性地测试 NoPE 与 RoPE 的表现。

Problem (no_pos_emb): Implement NoPE (0.5 B200 hrs) (1 point)

修改你带 RoPE 的 Transformer 实现，完全移除 position embedding 信息，并观察会发生什么。
交付内容：比较 RoPE 和 NoPE 性能的 learning curve。

消融 3：SwiGLU vs. SiLU

接下来，我们将遵循 N. Shazeer [20]，通过比较 SwiGLU feed-forward networks 与使用 SiLU activations 但没有 gated linear unit（GLU）的 feed-forward networks，测试 feed-forward network 中 gating 的重要性：

$$\text{FFN}_{\text{SiLU}}(x) = W_2\text{SiLU}(W_1 x). \tag{29}$$

回忆一下，在我们的 SwiGLU 实现中，内部 feed-forward layer 的维度约为 $d_{ff} = \frac{8}{3}d_{model}$（同时确保 $d_{ff} \bmod 64 = 0$，以利用 GPU tensor cores）。在这个 ablation baseline 中，你的 FFN_{SiLU} 实现应改为设置 $d_{ff} = 4 \times d_{model}$，以大致匹配默认 SwiGLU feed-forward network 的参数数量（默认 SwiGLU 有三个而不是两个 weight matrices）。

Problem (swiglu_ablation): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point)

交付内容：比较 SwiGLU 和 SiLU feed-forward networks 性能的 learning curve，参数数量应大致匹配。
交付内容：几句话讨论你的发现。

Low-Resource Tip: Online students with limited GPU resources should test modifications on TinyStories

在本作业余下部分，我们将转向更大规模、噪声更大的 web dataset（OpenWebText），实验架构修改，并（可选地）向课程 leaderboard 提交。
在 OpenWebText 上训练一个 LM 到流畅需要很长时间，因此我们建议 GPU 访问受限的在线学生继续在 TinyStories 上测试修改（使用 validation loss 作为评估性能的 metric）。

7.4 在 OpenWebText 上运行

现在我们将转向一个由 web crawl 创建的更标准预训练数据集。OpenWebText [A. Gokaslan et al., 2019] 的一个小样本也作为单个文本文件提供：如何访问该文件见第 1 节。

下面是 OpenWebText 中的一个示例。注意文本更加真实、复杂且多样。你可能想浏览训练数据集，以了解 web-scraped corpus 的训练数据是什么样的。

示例 (owt_example)：OWT 中的一个示例

Baseball Prospectus 的技术总监 Harry Pavlidis 在雇用 Jonathan Judge 时冒了一个险。
Pavlidis 知道，正如 Alan Schwarz 在 *The Numbers Game* 中所写，“美国文化中没有哪个角落比棒球运动员的表现被计数得更精确、被量化得更热情。”只需点几下，你就可以发现 Noah Syndergaard 的 fastball 在到达本垒板的路上每分钟旋转超过 2,100 次；Nelson Cruz 在 2016 年合格 hitters 中拥有联盟最高的平均 exit velocity；以及无数其他看起来像从电子游戏或科幻小说中撕下来的小知识。不断上升的数据海洋赋能了棒球文化中一个日益重要的角色：分析型业余爱好者。
这种赋能也带来了更多审视——不仅针对测量本身，也针对背后的人和出版物。在 Baseball Prospectus，Pavlidis 深知定量不完美会伴随怎样的反弹。他也知道该网站的 catching metrics 需要重做，而要完成这项工作，需要一个博学的头脑——一个能够解决复杂统计建模问题的人。
“他让我们感到震惊。”Harry Pavlidis
Pavlidis 根据 Judge 的写作以及他们在一个由网站赞助的球场活动中的互动，直觉认为 Judge “懂了”。[…]

注意：你可能需要为这个实验重新调优 learning rate 或 batch size 等超参数。

Problem (main_experiment): Experiment on OWT (2 B200 hrs) (2 points)

在 OpenWebText 上训练你的语言模型，使用与 TinyStories 相同的模型架构和总训练迭代次数。这个模型表现如何?
交付内容：你的 OpenWebText 语言模型的 learning curve。描述其 losses 与 TinyStories 的差异——我们应如何解释这些 losses?
交付内容：与 TinyStories 输出相同格式的 OpenWebText LM generated text。该文本的流畅性如何? 为什么尽管我们使用与 TinyStories 相同的模型和计算预算，输出质量仍更差?

7.5 你自己的修改 + leaderboard

恭喜你走到这里。你差不多完成了！现在你将尝试改进 Transformer 架构，并看看你的超参数和架构与班上其他学生相比表现如何。

leaderboard 规则

除了以下限制外，没有其他限制：

运行时间：你的提交最多可以在 B200 上运行 45 分钟。如果你使用 SLURM 或 Modal，可能需要在提交脚本中强制执行这一点。

数据：你只能使用我们提供的 OpenWebText 训练数据集。

除此之外，你可以随心所欲地做任何事情。

如果你正在寻找一些可以实现的想法，可以查看以下资源：

- 最先进的开源 LLM families，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。
- NanoGPT speedrun repository（`github.com/KellerJordan/modded-nanogpt`），社区成员会在其中发布许多有趣的修改，用于“speedrunning”小规模语言模型预训练。例如，一个可追溯到原始 Transformer 论文的常见修改，是将 input 和 output embeddings 的 weights 绑定在一起（见 A. Vaswani et al. [8] 第 3.4 节，以及 A. Chowdhery et al. [16] 第 2 节）。如果你尝试 weight tying，可能需要降低 embedding/LM head init 的 standard deviation。

在尝试完整 45 分钟运行之前，你会想先在 OpenWebText 的一个小子集或 TinyStories 上测试这些想法。

作为一个 caveat，我们也注意到，你可能会发现某些在这个 leaderboard 中表现良好的修改，并不一定能泛化到更大规模的预训练。我们将在课程的 scaling laws 单元进一步探讨这个想法。

Problem (leaderboard): Leaderboard (10 B200 hrs) (6 points)

你将在上述 leaderboard 规则下训练一个模型，目标是在 0.75 B200-hours 内最小化语言模型的 validation loss。

交付内容：记录的最终 validation loss、一条清晰显示 wall-clock-time x-axis 且小于 45 分钟的相关 learning curve，以及你所做工作的描述。我们期望 leaderboard submission 至少优于 5.0 loss 的 naive baseline。提交到 leaderboard：`github.com/stanford-cs336/assignment1-basics-leaderboard`。

参考文献

[1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.

[2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.

[3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in Proc. of ACL, 2016.

[4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.

[5] P. Gage, "A new algorithm for data compression," C Users Journal, vol. 12, no. 2, pp. 23–38, Feb. 1994.

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners." 2019.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training." 2018.

[8] A. Vaswani et al., "Attention is All you Need," in Proc. of NeurIPS, 2017.

[9] T. Q. Nguyen and J. Salazar, "Transformers without Tears: Improving the Normalization of Self-Attention," in Proc. of IWSWLT, 2019.

[10] R. Xiong et al., "On Layer Normalization in the Transformer Architecture," in Proc. of ICML, 2020.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization." 2016.

[12] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models." 2023.

[13] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in Proc. of NeurIPS, 2019.

[14] A. Grattafiori et al., "The Llama 3 Herd of Models." [Online]. Available: https://arxiv.org/abs/2407.21783

[15] A. Yang et al., "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.

[16] A. Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways." 2022.

[17] D. Hendrycks and K. Gimpel, "Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units." 2016.

[18] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/1702.03118

[19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language Modeling with Gated Convolutional Networks." [Online]. Available: https://arxiv.org/abs/1612.08083

[20] N. Shazeer, "GLU Variants Improve Transformer." 2020.

[21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding." 2021.

[22] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. of ICLR, 2015.

[23] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proc. of ICLR, 2019.

[24] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. of NeurIPS, 2020.

[25] J. Kaplan et al., "Scaling Laws for Neural Language Models." 2020.

[26] J. Hoffmann et al., "Training Compute-Optimal Large Language Models." 2022.

[27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," in Proc. of ICLR, 2020.

[28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, "Transformer Dissection: An Unified Understanding for Transformer's Attention via the Lens of Kernel," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International

Joint Conference on Natural Language Processing (EMNLP-IJCNLP), K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: 10.18653/v1/D19-1443.

[29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, "The Impact of Positional Encoding on Length Generalization in Transformers," in Thirty-seventh Conference on Neural Information Processing Systems, 2023. [Online]. Available: https://openreview.net/forum?id=Drrl2gcjzl